Sistemas Multiagentes para Reestruturação Autônoma de Códigos

Análise de dependências, paralelização OpenMP e coordenação baseada em evidências

Lucas Nory Ulson — Ciência da Computação, PUCRS

Revisão multiplataforma de 5 de novembro de 2026, com protocolo de 15 medições e 3 aquecimentos. Texto alinhado à implementação, à campanha final do Mac e aos documentos recebidos. A campanha original do Coordinator terminou INCOMPLETE; o contrato foi corrigido e a replicação com as mesmas evidências terminou COMPLETE, mantendo ambos os resultados; esta versão ainda não constitui monografia final aprovada.

Resumo

Este trabalho avalia um protótipo que combina análise estática, transformação OpenMP e decisão de uma LLM sobre evidências. O caminho integrado recebe uma fonte C de redução escalar, gera um candidato por regras determinísticas, verifica sua corretude e mede configurações antes de permitir promoção. Uma nova campanha Linux reuniu 900 medições externas em 60 configurações, com três aquecimentos e quinze amostras por configuração. Foram avaliados Dot Product, GEMM, Nussinov, Jacobi-2D e Covariance, separando Dot C automaticamente gerado de Dot C++ manual e das variantes manuais dos demais casos. Nas entradas grandes, as configurações recomendadas alcançaram ganhos contra o sequencial de 1,254× no Dot C, 2,845× em GEMM, 3,195× em Nussinov manual, 1,177× em Jacobi-2D e 2,193× em Covariance. Dot C pequeno e Dot C++ menor não admitiram configuração OpenMP benéfica. Mediana, dispersão, MAD, IQR e intervalos bootstrap condicionais acompanham os resultados. O acervo Apple M5 foi preservado, e uma nova coleta com o mesmo protocolo foi preparada para aquela máquina. Na avaliação anterior do Coordinator no M5, doze respostas válidas reproduziram a política determinística nos casos estudados. Não foi demonstrado benefício adicional da LLM nessa amostra; os ganhos dos kernels pertencem às variantes e configurações medidas. As conclusões distinguem integração de software, escalabilidade e decisão, com limites explícitos de corretude empírica, variabilidade de execução e comparação entre plataformas completas.

Palavras-chave: OpenMP; análise de dependências; paralelização automática; escalabilidade; modelos de linguagem; reprodutibilidade experimental.

Abstract

This work evaluates a prototype combining static analysis, OpenMP transformation and an LLM decision over measured evidence. Its integrated path supports a C scalar reduction, produces a candidate through deterministic rules, checks correctness and measures configurations before promotion. A new Linux campaign collected 900 external observations across 60 configurations, with three warm-up runs and fifteen samples per configuration. Dot Product, GEMM, Nussinov, Jacobi-2D and Covariance were evaluated, distinguishing automatically generated Dot C from manual Dot C++ and manual variants of the other kernels. For large inputs, recommended configurations achieved total speedups of 1.254× for Dot C, 2.845× for GEMM, 3.195× for manual Nussinov, 1.177× for Jacobi-2D and 2.193× for Covariance. Small Dot C and the smaller manual Dot C++ input had no beneficial OpenMP configuration. Medians, dispersion, MAD, IQR and conditional bootstrap intervals accompany the measurements. Historical Apple M5 records were preserved, and an equivalent new protocol was prepared for that platform. In the earlier M5 Coordinator evaluation, twelve valid LLM responses reproduced the deterministic policy on the studied cases, without demonstrated additional decision benefit. Kernel speedups are therefore attributed to measured variants and configurations. The conclusions distinguish software integration, scaling and decision making, with explicit limits on empirical correctness, run-to-run variability and comparisons of whole platforms.

Keywords: OpenMP; dependence analysis; automatic parallelization; scaling; language models; experimental reproducibility.

SUMÁRIO

1. Introdução 6

1.1. Problema e questões de pesquisa 6

1.2. Objetivos e contribuições 7

2. Fundamentação e posicionamento 7

2.1. Dependências e padrões 7

2.2. Arquitetura por papéis e LLM 8

2.3. Avaliação de código paralelo 8

3. Arquitetura e implementação 9

3.1. Fluxo implementado 9

3.2. Recorte de automação 10

3.3. Contrato de evidências 10

3.4. Entradas, saídas e limites de cada componente 11

3.5. Algoritmos determinísticos e transformação suportada 13

3.6. Decisão da LLM e publicação controlada 15

3.7. Algoritmos dos kernels e diferença entre paralelismos 16

3.8. Procedimento de apresentação institucional 18

4. Metodologia 19

4.1. Unidades experimentais e proveniência 19

4.2. Corretude e compilação 19

4.3. Desempenho e estatística descritiva 20

4.4. Avaliação do Coordinator 21

4.5. Coleta complementar Linux e comparação multiplataforma 21

4.6. Protocolo principal multiplataforma de quinze medições 23
4.7. Ambientes Linux e Apple M5 25
5. Resultados disponíveis 26
5.1. Verificação técnica 26
5.2. Dot Product verificável no Mac 26
5.3. Casos históricos: Wavefront e Nussinov 27
5.4. Avaliação pareada do Coordinator no Mac 28
5.5. Campanha exploratória Linux: sete medições por configuração 29
5.6. Dot C exploratório: cinco medições e efeito do regime de medição 32
5.7. Comparação contextual com os registros do M5 33
5.8. Campanha principal Linux: três aquecimentos e quinze medições 34
6. Discussão e ameaças à validade 39
6.1. Interpretação técnica da escalabilidade 41
6.2. O que a campanha principal acrescenta 42
7. Conclusão compatível com as evidências atuais 44
Referências verificadas para esta revisão 45
Apêndice de rastreabilidade e atualização 46

1. Introdução

A reestruturação de um programa para execução paralela envolve decisões sobre dependências, distribuição do trabalho e configuração de execução. Neste trabalho, o problema é tratado como uma sequência de decisões verificáveis: identificar uma transformação suportada, testar o resultado e medir se uma configuração oferece ganho. A presença de diretivas paralelas, isoladamente, não é adotada como critério de sucesso.

OpenMP fornece construções de execução e de compartilhamento de dados para programação paralela. Sua especificação é a referência conceitual para os mecanismos empregados; isso não implica que os compiladores avaliados implementem integralmente todas as funcionalidades da versão 5.2. OpenMP ARB, 2021 (https://www.openmp.org/spec-html/5.2/openmp.html).

O uso de modelos de linguagem acrescenta uma decisão textual ao fluxo. Essa decisão pode interpretar informações produzidas pelas ferramentas, mas não substitui as medições nem a validação do candidato. Para investigar essa separação, o protótipo concentra a LLM no Coordinator e mantém as etapas de geração suportada, medição e admissibilidade como componentes determinísticos.

A proposta inicial descrevia quatro agentes, LangGraph e um ciclo de reparo mais amplo. A implementação avaliada adotou um recorte menor: orquestração em Python, evidências serializadas em JSON e um Coordinator LLM com ações estruturadas. A geração automática integrada foi demonstrada para Dot Product com redução escalar. Outros padrões aparecem nas análises e nas campanhas de variantes, com cobertura parcial de transformação. Essa distinção determina o alcance das conclusões.

1.1. Problema e questões de pesquisa

A pergunta central do recorte é: como integrar análise, transformação suportada, validação, medição e decisão de uma LLM preservando a rastreabilidade do candidato, e qual efeito adicional a LLM exerce sobre a decisão de admissão?

Três questões organizam a avaliação:

1. R1 — Integração: como o pipeline vincula código, validação, configuração e medição, e em quais situações bloqueia a promoção?

2. R2 — Desempenho: quais configurações medidas apresentam ganho contra o sequencial e escalabilidade em relação à mesma variante OpenMP com uma thread?

3. R3 — Coordinator: mantendo candidato e evidências constantes, o Coordinator altera decisões e resultados admitidos em relação à política determinística, e qual é seu custo em latência e chamadas?

R3 não equivale a comparar geração one-shot com geração por vários agentes. Também não mede redução de tentativas de uma busca autônoma por novos códigos, pois tal busca não é executada pelo runner atual.

1.2. Objetivos e contribuições

O objetivo geral é desenvolver e avaliar, no recorte suportado, um pipeline híbrido de reestruturação OpenMP com decisão baseada em evidências. Os objetivos específicos são extrair informações estruturais; planejar e aplicar uma transformação implementada; verificar saídas; medir configurações; rastrear os artefatos utilizados; e comparar a decisão da LLM com a política sem LLM.

As contribuições técnicas demonstradas são a integração do caminho de redução escalar, o contrato que vincula evidências ao candidato, a revalidação antes da publicação e a consolidação de métricas com baselines distintos. A contribuição empírica adicional do Coordinator depende da conclusão de R3; não é pressuposta pela existência desses componentes.

2. Fundamentação e posicionamento

2.1. Dependências e padrões

O analisador estático utiliza o AST produzido pelo Clang para extrair informações sobre laços, acessos e atualizações. O analisador de dependências e o classificador aplicam regras implementadas sobre essa representação. Seus resultados orientam o planejamento, sem constituir uma análise completa de todos os recursos de C, de aliasing arbitrário ou de bibliotecas externas.

Os casos do repositório são organizados em DOALL, REDUCTION, STENCIL, WAVEFRONT/DOACROSS e MIXED. Essa taxonomia de padrões não deve ser confundida com uma classificação da arquitetura física de execução. Em particular, distribuir iterações entre threads e inserir uma redução SIMD específica são operações diferentes no protótipo.

2.2. Arquitetura por papéis e LLM

O termo multiagente é usado no projeto para organizar responsabilidades especializadas. No caminho avaliado, a maioria desses papéis é implementada por módulos determinísticos; um Coordinator utiliza uma LLM. Não foi implementado um conjunto de modelos independentes negociando planos, um motor BDI ou uma memória blackboard com atualização concorrente. O contexto compartilhado é materializado por artefatos e referências explícitas entre etapas.

Como referência de arquitetura, AutoGen apresenta composição de agentes capazes de conversar e combinar modelos, ferramentas e intervenção humana. O presente protótipo não utiliza esse framework nem avalia o mesmo mecanismo de conversação. Essa referência situa a diferença entre uma arquitetura multiagente conversacional e a divisão de papéis adotada aqui. Wu et al., 2023 (https://arxiv.org/abs/2308.08155).

2.3. Avaliação de código paralelo

ParEval foi proposto para avaliar geração de código paralelo por LLMs sobre 420 tarefas, considerando diferentes tipos de problema e modelos de programação. Esse trabalho relaciona o tema de LLMs à avaliação de código paralelo. O estudo atual possui outra unidade experimental: a decisão sobre um candidato já gerado e medido. Não se compara numericamente o resultado deste TCC aos resultados de ParEval. Nichols et al., 2024 (https://arxiv.org/abs/2401.12554).

Parte dos kernels históricos deriva de PolyBench/C, uma coleção de benchmarks com instrumentação e inicialização de dados para avaliação de kernels. O projeto fixa um submódulo PolyBenchC-4.2.1 para esses casos; os fixtures independentes de Dot Product permitem

exercitar o novo contrato sem essa dependência. Pouchet, PolyBench/C (https://www.cs.colostate.edu/~pouchet/software/polybench/polybench.html).

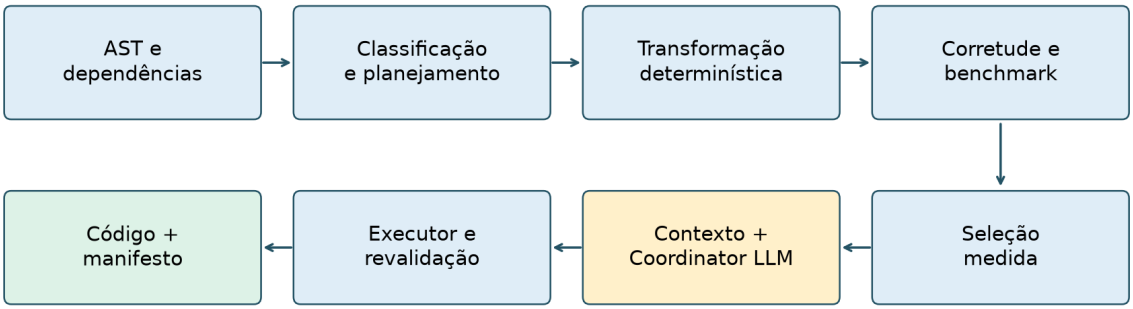
Essas referências fundamentam o recorte, mas não constituem uma revisão sistemática da literatura nem demonstram novidade absoluta da arquitetura.

3. Arquitetura e implementação

3.1. Fluxo implementado

O runner executa análise estática, dependências, classificação, planejamento, transformação, gate de corretude, gate de desempenho, seleção, construção do contexto, Coordinator e promoção. O modo de preparação encerra após construir o contexto verificado, permitindo separar a coleta determinística da inferência.

Figura 1 — Arquitetura implementada: componentes determinísticos, Coordinator LLM e publicação



Fonte: elaboração a partir do código e dos resultados do projeto.

Etapa	Responsabilidade	Implementação principal
Análise	Extrair estrutura e relações reconhecidas	analysis/static_analyzer.py, dependence_analyzer.py
Classificação e plano	Identificar padrão e transformação implementada	pattern_classifier.py, strategy_planner.py
Transformação	Aplicar redução escalar direta suportada	analysis/code_transformer.py
Corretude	Compilar e comparar saída numérica nas threads solicitadas	scripts/compile_correctness_gate.py
Desempenho	Registrar tempos brutos e resumos	scripts/performance_gate.py
Seleção	Escolher configuração a partir de medições	analysis/performance_selector.py

Etapa	Responsabilidade	Implementação principal
Contexto e decisão	Agregar evidências e obter ação estruturada	agents/context_builder.py, coordinator_agent.py
Promoção	Conferir evidências, revalidar e publicar	scripts/promote_candidate.py

A implementação do planejador adaptativo e as transformações específicas de Nussinov são caminhos adicionais. Sua existência não significa que o runner de fontes independentes execute automaticamente todo o projeto PolyBench.

3.2. Recorte de automação

Caso	O que foi automatizado / avaliado
Dot Product C	Caminho integrado de redução escalar, gates, seleção, decisão e executor; inferência real e promoção concluídas no Mac em 03/11/2026
GEMM	Análises e campanhas de variante OpenMP manual
Jacobi-2D	Análises, variante OpenMP manual e experiências específicas com tiling
Covariance	Análises e campanha de variante OpenMP manual
Nussinov	Análises e transformações adaptativas específicas sobre base OpenMP existente; contratos legados nas campanhas
Wavefront sintético	Diagnóstico de baixo desempenho, variante tiled manual e autotuning

A classificação de um padrão não comprova a geração automática de código para esse padrão. O caso Wavefront registra uma trajetória experimental de diagnóstico, modificação e autotuning; os artefatos não comprovam que a LLM executou esse ciclo de forma autônoma.

3.3. Contrato de evidências

Os gates atuais preservam cópias dos arquivos explícitos de entrada e registram hashes SHA-256 de fontes e executáveis, identidade do compilador, flags, variáveis OpenMP, campanha e tempos individuais. A seleção é recalculada ao carregar a evidência. O contexto inclui o código verificado e referências aos relatórios. A decisão registra a identidade do contexto recebido.

O vínculo de hashes permite detectar a troca de um arquivo após a medição. Não é uma assinatura de um produtor externo, nem prova que um relatório fabricado fora do processo é verdadeiro. Também não captura integralmente bibliotecas, headers de projeto ou todos os

componentes de uma toolchain.

O Coordinator emite ACCEPT, RETUNE, MODIFY ou REJECT e uma próxima ação compatível. RETUNE/MODIFY/REJECT encerram a tentativa do runner sem publicar um artefato. ACCEPT não basta para promoção: o executor exige configuração medida, ganho contra ambos os baselines e revalidação fresca de corretude. A publicação é feita por staging e renomeação do diretório final, com código, manifesto e evidências. Uma tentativa duplicada é recusada.

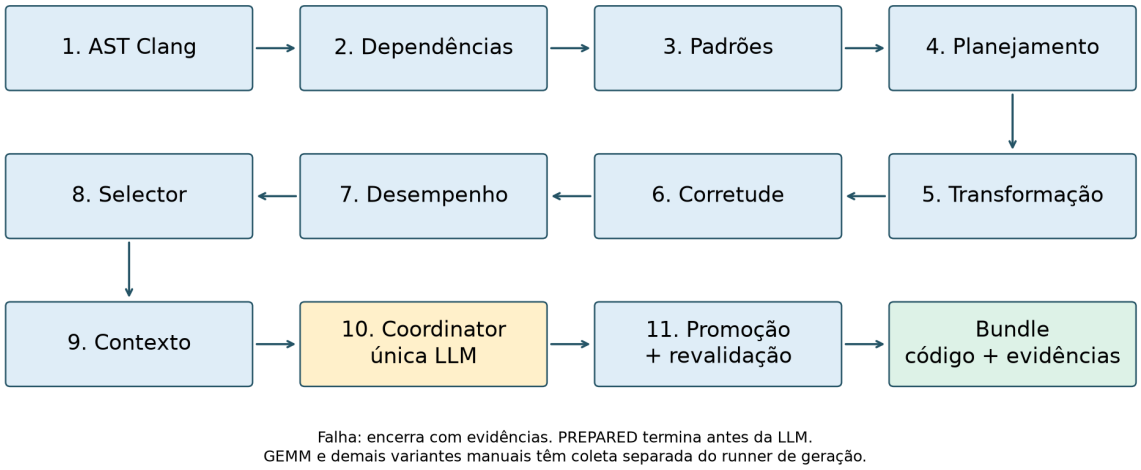
3.4. Entradas, saídas e limites de cada componente

O runner é um programa Python que chama outros programas e troca arquivos JSON. Não há escalonamento concorrente entre agentes nesse caminho. O termo agente designa um papel funcional, não uma instância independente de LLM. Apenas o Coordinator consulta um modelo de linguagem; análise, transformação e gates são determinísticos. Essa distinção determina o que cada resultado pode demonstrar.

Componente e função	Entrada efetiva	Saída consumida pela etapa seguinte	Limite que importa à interpretação
Static Analyzer: analyze, run_clang	Fonte, diretórios de include e AST JSON do Clang	Funções kernel, laços, intervalos de linhas, leituras, escritas e atualizações escalares	Não é um compilador de otimização completo; depende do AST e das construções reconhecidas
Dependence Analyzer: analyze_loop, compare_accesses	Acessos extraídos do mesmo laço	Relações entre leitura/escrita do mesmo array e offsets reconhecidos	Não resolve aliasing arbitrário; compara candidatos leitura–escrita, sem prova geral de independência
Pattern Classifier: classify_loop, classify_kernel	Estrutura e relações de dependência	Família, evidências e rótulo heurístico de confiança	Reconhecer um padrão não demonstra que existe transformação implementada
Strategy Planner: plan_kernel, plan_reduction	Padrões e hierarquia de laços	Estratégia, alvo, variável, operador e autorização de transformação direta	Um plano DOALL ou WAVEFRONT não torna esses padrões suportados pelo transformador genérico
Code Transformer: apply_strategy, transform_scalar_reduction	Fonte e plano ligados à análise	Fonte com pragma e registro da transformação	Caminho genérico restrito a redução escalar direta; demais estratégias são recusadas
Correctness Gate: run_gate, validate_numeric	Original, candidato, driver e threads	Compilação, saídas finitas, tolerâncias e relatório v0.2	Validação empírica do driver, sem equivalência formal ou detector de races
Performance Gate: execute, parse_average_time	Evidência de corretude e driver de benchmark	Amostras e relatório v0.1 ligado à corretude	Protocolo original de cinco amostras; distinto da nova coleta de quinze

Componente e função	Entrada efetiva	Saída consumida pela etapa seguinte	Limite que importa à interpretação
Performance Selector: select_configuration	Tempos por configuração	Mais rápida, conjunto de quase empate e recomendação	Política determinística de 2%; não busca novo código ou configurações não medidas
Context Builder: build_context	Análises e evidências verificáveis	Contexto v0.3, bytes congelados e referências	Uma declaração PASS externa não substitui verificação da cadeia de evidências
Coordinator: compact_context, call_ollama, validate_decision	Prompt, schema e contexto	Decisão v0.2 e metadados da inferência	Não gera os kernels nem executa reparo; decisões inválidas são recusadas
Promotion Executor: promote_candidate	ACCEPT válido e evidências	Bundle, manifesto e revalidação	Só publica a configuração admissível e medida, após nova compilação e corretude

Figura 2 — Componentes e artefatos do caminho integrado



Fonte: elaboração a partir do código e dos resultados do projeto.

As setas da figura representam dependência entre artefatos e ordem do runner. Falhas de compilação, ausência de evidência ou divergência de hash encerram a tentativa. O modo --prepare-only termina antes da inferência e da promoção. Na campanha multiplataforma, um harness de medição separado reutiliza os cinco primeiros componentes para gerar Dot C e valida as variantes de todos os kernels. Ele não apresenta seu arquivo experimental como um bundle promovido.

3.5. Algoritmos determinísticos e transformação suportada

Os pseudocódigos abaixo resumem funções existentes. Os nomes de arquivo e de função permitem conferir o comportamento no repositório; os blocos não são novos algoritmos implementados apenas para a redação.

Algoritmo 1 — extração estrutural e dependências (static_analyzer.analyze e dependence_analyzer.analyze_loop).

```
receber fonte e diretórios de include
executar Clang com AST em JSON e verificar seu retorno
encontrar funções cujo nome identifica um kernel
para cada kernel e laço reconhecido:
    registrar localização, iterador e hierarquia
    coletar leituras, escritas e atualizações escalares
    para cada escrita w e leitura r do mesmo array:
        comparar índices reconhecidos em w e r
        registrar relação e offsets, quando obtidos
gravar análises em arquivos distintos por fonte/variante
```

O comparador trata expressões simples de índice e distingue mesma localização, acesso deslocado e relação simbólica. Uma lista vazia de dependências significa que o procedimento não encontrou candidatos segundo suas regras; não é um certificado universal de independência. Ponteiros que se sobrepõem, chamadas externas e expressões não reconhecidas exigem cautela.

Algoritmo 2 — classificação e autorização (classify_loop, classify_kernel, plan_reduction e apply_strategy).

```
classificar laços a partir das evidências extraídas
    acessos deslocados sustentam candidato wavefront
    atualização escalar reconhecida sustenta candidato reduction
    demais regras identificam redução em array, stencil e doall
agregar as classificações no kernel e construir a hierarquia
produzir plano com laço, variável, operador e construção OpenMP
se a estratégia não for REDUCTION: recusar no caminho genérico
para cada redução, da última linha da fonte para a primeira:
    exigir tipo scalar e direct_code_transformation autorizado
    localizar o laço e conferir a linha de inserção
    inserir pragma parallel for reduction(...) schedule(static)
gravar candidato e metadados da transformação
```

O tratamento em ordem inversa evita que uma inserção desloque as posições dos próximos alvos. O rótulo de confiança do classificador é uma classificação heurística do programa, não uma probabilidade calibrada de correção. GEMM não passa a ser automaticamente transformado porque seu laço foi chamado de DOALL.

O exemplo a seguir é a função sequencial real de analysis/fixtures/dot_product.c, apenas compactada na apresentação:

```
double kernel_dot(int n, const double *A, const double *B) {
    double sum = 0.0;
    for (int i = 0; i < n; i++) {
        sum += A[i] * B[i];
    }
    return sum;
}
```

O candidato gerado na campanha mantém a função e acrescenta a redução:

```
double kernel_dot(int n, const double *A, const double *B) {
    double sum = 0.0;
    #pragma omp parallel for reduction(+:sum) schedule(static)
    for (int i = 0; i < n; i++) {
        sum += A[i] * B[i];
    }
    return sum;
}
```

A variável acumuladora é privada segundo a semântica de redução e seus valores parciais são combinados ao final. A associação das operações pode mudar; por isso, a validação floating-point usa tolerância explícita em vez de exigir a mesma sequência de bits do sequencial. O resultado continua sujeito às entradas e ao comportamento efetivamente testados. OpenMP 5.2, reduction (https://www.openmp.org/spec-html/5.2/openmpsu52.html).

Algoritmo 3 — corretude e vínculo de evidências (run_gate e load_selection_evidence).

```
capturar original, candidato, driver e identidade do compilador
calcular SHA-256 e criar identidade exclusiva da campanha
compilar os snapshots sequencial e OpenMP
se houver erro de compilação ou execução: FAIL
obter escalar de referência e exigir valor finito
para cada configuração de threads solicitada:
    executar candidato e exigir escalar finito
    comparar com referência usando rtol=1e-9 e atol=1e-12
    se falhar: conservar o relatório e encerrar com FAIL
emitir PASS com fontes, executáveis, parâmetros e resultados
ao consumir a evidência: verificar hashes e a mesma campanha
```

O gate integrado testa o driver fornecido. O novo harness amplia a cobertura de entrada da campanha e compara dumps completos dos casos PolyBench, mas não transforma esses testes em uma prova formal. Seu protocolo e suas saídas são descritos separadamente na metodologia.

Algoritmo 4 — seleção de desempenho (select_configuration).

```
localizar sequencial e OpenMP 1T da mesma variante
calcular speedup total por configuração medida
identificar configurações que superam o sequencial
se nenhuma superar: não recomendar candidato benéfico
localizar a menor mediana entre configurações benéficas
```

```
formar conjunto com tempo até 1,02 vezes a menor mediana
recomendar a configuração de menos threads nesse conjunto
registrar separadamente transformação e ganho paralelo
```

O Selector pode reconhecer benefício já em uma thread; isso é transformação, não escalabilidade. O executor impõe condições adicionais: recomendação paralela com pelo menos duas threads, ganho contra os dois baselines e evidências íntegras. A mediana mais baixa e a recomendação são mantidas separadas nas tabelas.

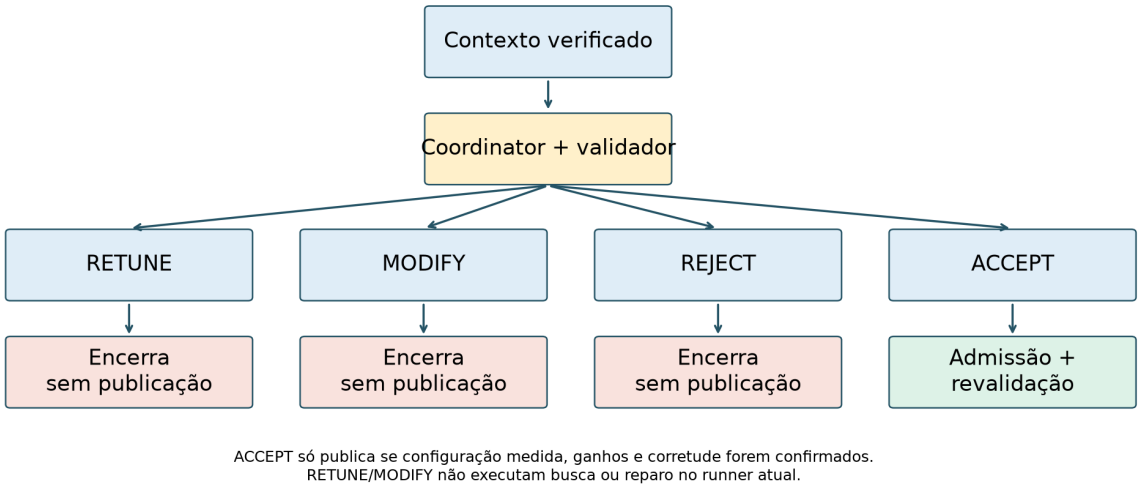
3.6. Decisão da LLM e publicação controlada

build_context carrega a seleção e os gates ligados à mesma campanha, verifica as referências e recupera os bytes congelados da fonte. Não toma uma string PASS como prova suficiente. compact_context mantém código, padrão, plano, dependências, validação, desempenho e restrições para construir o prompt.

Algoritmo 5 — decisão e promoção (run_pipeline, validate_decision e promote_candidate).

```
construir contexto a partir de evidências verificadas
se prepare_only: retornar PREPARED antes da LLM
consultar Ollama com prompt e schema estruturado
validar decisão, ação e configuração contra o contexto
se RETUNE, MODIFY ou REJECT:
    registrar decisão e encerrar sem publicar
se ACCEPT e PROMOTE_CANDIDATE:
    conferir identidades do contexto, candidato e campanha
    exigir configuração medida com ganho vs. sequencial e OpenMP 1T
    reservar destino exclusivo e criar staging
    recompilar e repetir corretude na configuração selecionada
    se houver falha ou alteração: recusar publicação
    arquivar código, evidências e manifesto
    publicar por renomeação atômica
```

Figura 3 — Decisão, encerramento e condições adicionais de publicação



Fonte: elaboração a partir do código e dos resultados do projeto.

RETUNE e MODIFY expressam intenções na interface, mas o runner avaliado não executa uma nova busca nem chama um codificador para reparar o candidato. Na avaliação pareada, a recusa usa fallback sequencial; esse fallback é uma regra do experimento, não um laço oculto de recuperação do runner. Os timeouts e respostas inválidas são observações, não tentativas apagadas.

O manifesto da promoção registra a configuração medida, métricas e hashes. SHA-256 permite detectar troca de bytes em relação aos registros; não é uma assinatura externa nem garante honestidade de relatórios fabricados fora do processo. A revalidação cobre a entrada do driver, não todos os programas C.

3.7. Algoritmos dos kernels e diferença entre paralelismos

No GEMM, cada linha de C pode ser calculada por uma thread. O excerto de kernels/gemm/manual_openmp.c conserva a ordem i–k–j e o contrato C ← αAB + βC:

```
#pragma omp parallel for private(j, k) schedule(static)
for (i = 0; i < ni; i++) {
    for (j = 0; j < nj; j++) C[i][j] *= beta;
    for (k = 0; k < nk; k++)
        for (j = 0; j < nj; j++)
            C[i][j] += alpha * A[i][k] * B[k][j];
}
```

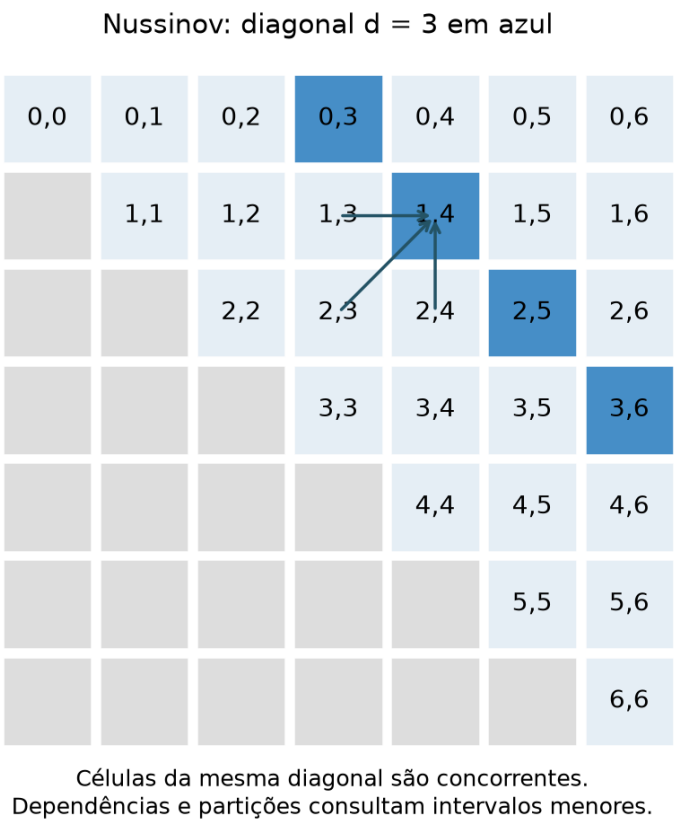
A partição por i evita que duas threads atualizem a mesma linha de C. O laço j acessa posições contíguas, enquanto B é reutilizada. O programa

avaliado é uma implementação simples, sem atribuir-lhe a eficiência de uma BLAS otimizada.

Nussinov calcula uma tabela de programação dinâmica sobre intervalos da sequência. A fonte original percorre linhas; a variante OpenMP manual usa diagonais d=j−i. O pseudocódigo conserva a recorrência implementada:

```
inicializar sequência sintética e tabela com zeros
criar uma região paralela persistente
para d = 1 até N-1, em todas as threads:
    distribuir i = 0 até N-d-1 com omp for schedule(static)
        j = i + d
        incorporar table[i][j-1] e table[i+1][j]
        incorporar table[i+1][j-1] e pareamento, se não adjacentes
        para k = i+1 até j-1:
            maximizar com table[i][k] + table[k+1][j]
    sincronizar antes de começar a próxima diagonal
```

Figura 4 — Dependências entre células e diagonais de Nussinov



Fonte: elaboração a partir do código e dos resultados do projeto.

As células da mesma diagonal são concorrentes, mas consultam diagonais anteriores. O fim do omp for precisa da barreira implícita. A largura disponível N−d diminui ao longo do cálculo; a carga e os acessos a intervalos também mudam com d. Isso difere do conjunto de

linhas independentes de GEMM. OpenMP 5.2, worksharing-loop (https://www.openmp.org/spec-html/5.2/openmpse66.html).

A adaptação SIMD substitui atualizações repetidas do destino por um acumulador local e uma redução de máximo no laço k. O trecho foi extraído da variante automática específica existente, não escrito pela LLM:

```
DATA_TYPE tcc2_best = table[i][j];
#pragma omp simd reduction(max:tcc2_best)
for (k = i + 1; k < j; k++) {
    DATA_TYPE tcc2_candidate = table[i][k] + table[k + 1][j];
    tcc2_best = max_score(tcc2_best, tcc2_candidate);
}
table[i][j] = tcc2_best;
```

Jacobi-2D atualiza dois arrays em passos de tempo. Os pontos de cada varredura podem ser distribuídos, mas as fases e os passos precisam respeitar suas dependências; não é correto paralelizar o eixo temporal sem outra transformação. Covariance contém três fases: médias por coluna, centralização dos dados e matriz de covariância. A versão manual usa barreiras entre fases e explora paralelismo nos índices externos. O domínio triangular da covariância pode desbalancear a distribuição estática de trabalho. Esses dois casos ampliam a avaliação de desempenho; não ampliam a cobertura do transformador genérico.

3.8. Procedimento de apresentação institucional

Esta revisão preserva a base original e a versão de trabalho anterior. O corpo é organizado em problema, fundamentação, implementação, metodologia, resultados, discussão e conclusão, com pseudocódigos e exemplos conferíveis. A capa oficial pública da Biblioteca PUCRS foi preservada e uma cópia preenchida é gerada com autor, escola, curso, título, cidade e ano. Os campos não fornecidos, como nome de orientador e composição da banca, não foram inventados.

O corpo de revisão utiliza A4, margens superior/esquerda de 3 cm e inferior/direita de 2 cm, fonte de texto de 12 pontos e entrelinhas de 1,5. Essas opções melhoram a apresentação, mas não equivalem à certificação de aderência integral ao modelo institucional atual, cujo acesso é autenticado. A paginação e a integração final da capa exigem conferência no aplicativo de edição utilizado na entrega. Biblioteca PUCRS, modelo e acesso (https://biblioteca.pucrs.br/apoio-a-pesquisa/modelos-de-normas-tecnicas-de-documentacao/trabalhos-academicos-abnt

/), capa de graduação (https://biblioteca.pucrs.br/apoio-a-pesquisa/capas-padrao-pucrs/capa-para-tcc/).

4. Metodologia

As subseções 4.1–4.4 descrevem o acervo e os protocolos anteriores. A coleta exploratória de sete amostras é descrita em 4.5. O protocolo principal de três aquecimentos e quinze medições externas está em 4.6, com os ambientes comparados em 4.7. Os conjuntos são analisados separadamente.

4.1. Unidades experimentais e proveniência

São separados três conjuntos de evidências. O primeiro contém 17 campanhas históricas recuperadas de CSVs, logs e dumps, sem o contrato atual completo de vínculo ao código medido. O segundo contém duas coletas Linux cujo resumo foi preservado, mas cujos contextos originais não estão neste checkout. O terceiro contém três campanhas finais de Dot Product verificadas pelos gates no Mac em 03/11. A consolidação totaliza 22 campanhas e 173 configurações, com 545 tempos individuais recuperados.

Os 35 CSVs históricos foram inventariados sem alteração, junto a nove CSVs das campanhas finais. Oito históricos apresentam linhas incompatíveis com o cabeçalho; vários acumulam mais de uma execução. A matriz usa fontes selecionadas explicitamente, conferindo resumos com amostras quando disponíveis. Não se unem amostras de máquinas ou campanhas diferentes para aumentar artificialmente o tamanho da amostra.

4.2. Corretude e compilação

No caminho independente, a corretude compara o resultado escalar do candidato ao sequencial com tolerâncias relativa de 1e-9 e absoluta de 1e-12. O driver determinístico de validação utiliza N=1.000.000. São verificadas as threads que serão medidas. Esse driver é distinto do driver de benchmark; portanto, o PASS não deve ser descrito como teste de todas as entradas temporizadas.

Nos casos legados PolyBench, há validações por comparação de dumps da compilação de validação e campanhas de desempenho com

instrumentação de tempo. Esses resultados têm cobertura e registro diferentes dos gates novos. O runner atual não inclui um detector de races nem estabelece equivalência formal.

Na campanha local foram usados GCC 13.3.0 para compilação e Clang para análise AST, em Linux x86_64 com Xeon E3-1225 V2 e quatro núcleos. Os flags de desempenho são -O3 para sequencial e -O3 -fopenmp para o candidato. O ambiente registra OMP_DYNAMIC=FALSE, OMP_PROC_BIND=TRUE e OMP_PLACES=cores. As campanhas históricas do Mac usam o ambiente Apple M5 registrado em setembro, com as lacunas específicas indicadas no catálogo.

4.3. Desempenho e estatística descritiva

O benchmark local possui dois tamanhos: N=10.000.000, com 100 chamadas internas, e N=1.024, com 10.000 chamadas internas. Cada configuração tem cinco medições externas; o tempo por chamada é reportado pelo driver. A medida principal é a mediana dessas cinco observações, mantendo média, mínimo, máximo e desvio padrão populacional para inspeção.

Se T_seq é o tempo mediano do sequencial, T_1 o tempo da variante OpenMP com uma thread e T_p o tempo dessa mesma variante com p threads, calculam-se:

- Ganho contra o sequencial: S_total = T_seq / T_p.

- Ganho da transformação: S_transf = T_seq / T_1.

- Escalabilidade paralela: S_par = T_1 / T_p.

- Eficiência paralela: E_par = S_par / p.

- Decomposição: S_total = S_transf × S_par.

No autotuning tiled, T_1 deve corresponder ao mesmo tamanho de bloco da configuração avaliada. O melhor bloco em uma thread pode ser diferente daquele selecionado em várias threads. Usar blocos distintos no quociente mistura os efeitos e não representa escalabilidade da mesma variante.

O Selector considera quase empates de até 2% e pode recomendar menos threads que a configuração de menor mediana. Os gráficos

mostram razões de medianas, sem intervalos de confiança ou testes de significância inventados. As amostras disponíveis não justificam generalização estatística entre plataformas.

4.4. Avaliação do Coordinator

O estudo pareado compara a política determinística, o Coordinator completo e o Coordinator com análises estruturadas retiradas do prompt. Os três modos recebem o mesmo candidato e as mesmas medições. A ablação preserva os gates e o contexto canônico de validação; o script verifica se o prompt mudou.

A ordem dos modos é embaralhada com semente 2026. O protocolo prevê três repetições de decisão por caso, temperatura zero e timeout explícito. Uma repetição da LLM não representa uma nova observação independente do tempo do kernel. Cada tentativa considera um candidato e, nos modos LLM, uma chamada; não há recuperação automática de novas variantes.

Registram-se decisões brutas e enriquecidas, erros, timeouts, concordância binária com a política, aceitação efetiva, propostas inadmissíveis, oportunidades elegíveis não aceitas, latência e tokens disponíveis. Concordância não é uma verdade independente sobre a melhor ação textual. Falhas permanecem no denominador. Custo monetário não é estimado sem dados.

O resultado admitido usa a configuração paralela elegível; recusa ou falha usa o sequencial como fallback definido para a avaliação. O tempo contabilizado soma o prefixo determinístico compartilhado ao custo de decisão/validação, sem representar nova execução independente do pipeline e sem incluir publicação.

4.5. Coleta complementar Linux e comparação multiplataforma

Em 05/11/2026 foi iniciada uma coleta complementar de escalabilidade no Xeon E3-1225 V2, com quatro núcleos físicos, Linux x86_64, Clang 18.1.3 e runtime OpenMP identificado nas saídas de vinculação. O governor observado foi schedutil; não houve fixação de frequência, isolamento exclusivo da máquina ou controle térmico. Não se executaram testes nem inferências junto à campanha. Usaram-se 1, 2, 3 e 4 threads, -O3, sem -march=native ou -ffast-math.

Foram selecionados Dot Product C++ com N=1.000.000 e 10.000.000, GEMM double com dimensões 200×220×240 e 1000×1100×1200, e Nussinov int com N=500 e 2500. As variantes preexistentes são manual no Dot/GEMM e manual, SIMD manual e SIMD automaticamente adaptada no Nussinov. Não houve inferência nem nova otimização manual dos kernels nessa campanha.

Cada configuração utiliza sete processos de medição e um processo prévio de aquecimento. A ordem é embaralhada em blocos completos, com semente 2026, preservando amostras individuais. O Dot C++ temporiza uma única chamada: a inicialização do runtime OpenMP pode estar incluída. Aquecimento em outro processo não conserva a equipe de workers. Portanto, esse Dot não substitui o experimento C do pipeline com 100 chamadas internas. PolyBench conserva seu flush de cache padrão antes de iniciar o timer. Os tempos excluem alocação, inicialização dos arrays, compilação e inferência. O flush OpenMP do utilitário PolyBench já cria a equipe antes do cronômetro de GEMM/Nussinov, diferentemente do Dot C++. O timer PolyBench usa gettimeofday com saída de seis casas decimais; manteve-se essa instrumentação preexistente.

A validação cobre cada combinação de entrada, variante e threads. Dot compara o escalar com a fórmula esperada; GEMM compara todos os elementos impressos com 17 dígitos significativos, em vez das duas casas decimais históricas; Nussinov compara o dump inteiro exato. Os executáveis de dump e tempo são separados e compilados com a mesma otimização. Os arquivos originais são preservados; a mudança de precisão ocorre no header copiado da campanha. Fontes, headers, utilitário PolyBench, comandos, bibliotecas, executáveis e saídas são arquivados com SHA-256. Essa coleta possui protocolo próprio e não autoriza promoção pelo contrato de fontes independentes do pipeline.

As figuras exibem mediana e amplitude mínimo–máximo, sem intervalos de confiança ou alegação de significância. Os CSVs novos também registram desvio padrão amostral; a consolidação histórica usa desvio padrão populacional. A campanha nova equivalente no M5 está preparada, mas não foi medida nesta máquina Linux. Dados históricos são contexto, sem permitir atribuir causalmente diferenças à arquitetura. Esta coleta amplia a avaliação de R2, sem ampliar a

amostra de decisões LLM de R3.

4.6. Protocolo principal multiplataforma de quinze medições

A coleta principal de 05/11 adota três execuções externas de aquecimento por configuração, seguidas de quinze medições externas. Linux mede sequencial e OpenMP 1T, 2T e 4T. O script M5 mantém códigos, entradas e critérios e acrescenta 6T, 8T e 10T; o trecho de comparação direta será 1T/2T/4T. Resultados da nova campanha M5 só serão acrescentados após execução real.

Caso compilado	Entrada menor	Entrada maior	Chamadas por processo
Dot C automático	N=1024	N=10.000.000	10.000 / 100
Dot C++ manual	N=1.000.000	N=10.000.000	1
GEMM	NI=200, NJ=220, NK=240	NI=1000, NJ=1100, NK=1200	1
Nussinov	N=500	N=2500	1
Jacobi-2D	N=250, TSTEPS=100	N=1300, TSTEPS=500	1
Covariance	N=260, M=240	N=1400, M=1200	1

São cinco algoritmos e seis casos compilados, pois Dot possui dois drivers explicitamente separados. Nussinov inclui manual, SIMD manual e adaptação SIMD específica preexistente. A matriz Linux contém 60 configurações e prevê 900 amostras externas e 180 execuções de aquecimento; o baseline de Nussinov é compartilhado, sem duplicação de observações.

Algoritmo 6 — coleta de benchmark_scaling.execute, acionada pelo wrapper benchmark_multiplatform.py.

```
reservar diretório exclusivo e registrar ambiente/Git/toolchain
compilar probe e conferir tamanhos reais das equipes OpenMP
gerar Dot C com os componentes determinísticos do projeto
para cada caso e entrada:
    congelar fontes, headers, driver e parâmetros efetivos
    compilar original e variantes OpenMP com a mesma otimização
    validar cada configuração contra referência/resultado esperado
    para cada um dos três blocos de aquecimento:
        embaralhar configurações e executar uma vez cada
    para cada um dos quinze blocos medidos:
        embaralhar configurações com o mesmo PRNG de seed 2026
        executar cada configuração e registrar stdout/stderr/retorno
        conferir resultado numérico do Dot e tempo finito positivo
        preservar observação e identidade do comando em JSONL
conferir fontes e executáveis por SHA-256 e emitir CSVs
em falha, conservar evidências e estado FAILED, sem nova tentativa oculta
```

Dot C é gerado e medido nesta campanha; não se reaproveitam os cinco tempos do Performance Gate original para completar quinze observações. Seu driver usa valores periódicos dependentes de i, enquanto o C++ usa dois vetores constantes. Mesmo quando N coincide, as duas execuções não isolam o efeito da linguagem ou da transformação. A identidade da fonte e do driver acompanha cada caso.

A ordem é aleatória reprodutível dentro de blocos completos. O gerador de seed 2026 também é usado nos aquecimentos antes das medições. Os registros permitem reconstruir a ordem efetiva. A medição de um kernel não ocorre junto de testes, renderização de figuras ou inferência iniciados nesta avaliação. A máquina não foi reservada exclusivamente, e serviços externos não são controlados.

São registrados média, mediana, desvio padrão amostral, mínimo, máximo, MAD sem fator de escala, Q1, Q3 e IQR. Os quartis usam interpolação linear inclusiva nas posições $(n-1)q$. Nenhuma amostra é excluída como outlier. Para $n \geq 15$, intervalos de 95% usam bootstrap percentil com 5.000 réplicas e seed 2026. Reamostram-se blocos completos: original, OpenMP 1T e p usam os mesmos índices de repetição, recalculando medianas e razões em cada réplica. O bloco é uma rodada do desenho pareado; não se utiliza moving-block bootstrap de rodadas temporais contíguas. A ideia de reamostragem da distribuição empírica é apresentada por Efron (1979) (https://sites.stat.washington.edu/courses/stat527/s13/readings/ann_stat1979.pdf).

Os limites são os percentis 2,5 e 97,5. Eles descrevem a incerteza condicional à campanha observada; não comprovam estacionariedade, independência temporal, superioridade causal de arquitetura ou ótimo global de threads. Não se aplicou correção de comparações múltiplas. A escolha posterior da menor mediana pode favorecer uma observação de sorte; a recomendação determinística é uma decisão operacional e não um teste estatístico de superioridade.

A corretude PolyBench usa dumps completos com 17 dígitos nos casos double e inteiros exatos em Nussinov, em executáveis separados dos temporizados. Fontes originais não são alteradas. Relógios e regimes de primeira chamada são os preexistentes, explicitados no protocolo. Código, comandos, entradas, logs, amostras, métodos estatísticos e figuras ficam em diretórios próprios.

4.7. Ambientes Linux e Apple M5

Característica	Linux da nova campanha	M5 do acervo histórico
Processador	Intel Xeon E3-1225 V2	Apple M5, Mac17,2
Arquitetura	x86-64	ARM64
Núcleos registrados	4 físicos e 4 lógicos	10, sendo 4 de desempenho e 6 de eficiência
Memória	19,47 GiB reportados pelo Linux, estado em environment.json	16 GB registrados em 03/11
Compilador	Clang 18.1.3	LLVM Homebrew 23.1.1 registrado em 03/11
Frequência/energia	Governor schedutil; frequência não fixada	Estado atual deve ser registrado na nova coleta
Threads da nova receita	1, 2 e 4	1, 2, 4, 6, 8 e 10

A coluna M5 descreve o acervo, não uma observação feita nesta execução Linux. O novo script registra novamente modelo, sistema, compilador, contagens de núcleos e energia. O valor histórico não é copiado como metadado de uma campanha futura. No Linux, caminho/hash dos compiladores, afinidade disponível, versões, macros de entrada e bibliotecas dinâmicas acompanham os registros.

A seleção de mais threads no M5 pode refletir maior capacidade disponível, mas também diferentes classes de núcleo, runtime, compilador e subsistema de memória. No wavefront, barreiras e redução de largura podem tornar novas threads mais caras que úteis. Em GEMM, a quantidade de trabalho por linha e a localidade são diferentes. Em Dot, movimento de dados e custos de entrada na região paralela podem dominar. Essas são hipóteses apoiadas na estrutura dos códigos, não causas isoladas por contadores ou intervenções neste experimento.

O comparador das duas campanhas novas exige a mesma identidade de fontes, headers, drivers, dimensões e protocolo, e mantém curvas separadas por plataforma. Diferenças incompatíveis impedem o gráfico conjunto. Mesmo quando esses critérios forem satisfeitos, a comparação será entre plataformas completas; não um teste que varia apenas quantidade de núcleos ou ISA.

5. Resultados disponíveis

5.1. Verificação técnica

A bateria final registrada contém 82 testes unittest e 14 casos de regressão dos seletores e Coordinator, cobrindo integração, configuração, alterações de artefatos, falhas de gates e promoção. Esses testes verificam comportamento de software. Respostas simuladas da LLM usadas em integração não são resultados científicos de inferência.

Uma execução de integração com análise Clang, compilação GCC/OpenMP e benchmark real publicou um bundle após resposta explicitamente simulada do Coordinator. O teste demonstra o mecanismo de publicação e revalidação, sem demonstrar que uma LLM real realizou a decisão dessa execução.

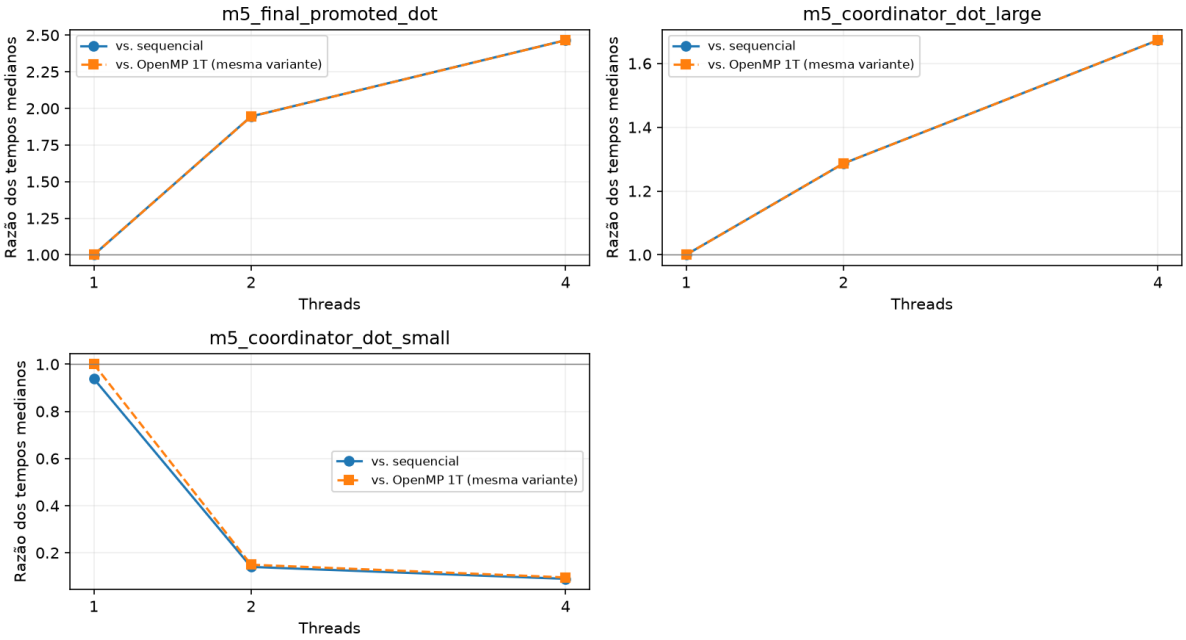
5.2. Dot Product verificável no Mac

Campanha	Sequencial (s)	OpenMP 1T (s)	Configuração selecionada	Resultado
Promoção final, N=10.000.000	0,004842180	0,004840610	4T: 0,001964010	Promovido
Estudo do Coordinator, N=10.000.000	0,007220030	0,007219770	4T: 0,004316330	Admite 4T
Estudo do Coordinator, N=1.024	0,000000799	0,000000852	—	Não admite paralelo

Na promoção final, o ganho contra o sequencial foi 2,465×; a escalabilidade contra OpenMP 1T também foi 2,465×, com eficiência paralela de aproximadamente 0,616. No estudo pareado, a entrada grande obteve 1,673× em 4T e a entrada pequena apresentou forte overhead paralelo. O resultado demonstra que o pipeline preserva uma recusa motivada pelas medições; não demonstra um ganho causado pelo Coordinator.

Figura 5 — Curvas verificáveis do Mac com baselines distintos

Apple M5 / 10 cores / 16 GB / macOS 26.6.2 (03/11) — campanhas separadas; sem agrupamento de amostras



Fonte: elaboração a partir do código e dos resultados do projeto.

Fonte: campanhas m5_final_promoted_dot, m5_coordinator_dot_large e m5_coordinator_dot_small na matriz consolidada. As medidas são por chamada; não são tempos totais de otimização nem repetições da decisão LLM.

5.3. Casos históricos: Wavefront e Nussinov

Campanha	Configuração analítica	Ganho vs. seq.	Escalabilidade da mesma variante	Eficiência paralela
Wavefront manual Mac, 09/09	1T	1,412×	1,000×	1,000
Wavefront tiled Mac, 09/09	bloco 64, 4T	3,400×	2,377×	0,594
Nussinov SIMD manual, 11/09	4T	4,037×	3,015×	0,754
Nussinov automático, 11/09	4T	4,106×	3,005×	0,751

No Wavefront manual, a recomendação de uma thread expressa benefício de transformação, sem sucesso multicore. A variante tiled apresenta ganho com quatro threads na campanha correspondente. Como os baselines sequenciais vêm de campanhas distintas, a comparação descreve a trajetória observada e não uma diferença pareada de medições ou uma contribuição causal da LLM.

No Nussinov automático, T_seq=0,858520 s, T_1=0,628325 s e T_4=0,209073 s. O ganho de transformação de 1,366× se combina à escalabilidade de 3,005×, produzindo aproximadamente 4,106× contra o sequencial. Dividir 4,106 por quatro não calcula a eficiência paralela. O hardware dessa campanha não está registrado na própria evidência canônica: a atribuição a Mac presente no resumo de outubro exige confirmação. A variante automática parte de base OpenMP manual.

As demais campanhas, incluindo GEMM, Jacobi e Covariance, constam da matriz e dos gráficos completos. Não se extrapola sua cobertura para transformação automática integral desses padrões.

5.4. Avaliação pareada do Coordinator no Mac

A campanha utilizou llama3.1:8b, ID local 46e0c10c039e, LLVM 23.1.1, threads 1, 2 e 4, três repetições e timeout de 600 segundos. As 12 chamadas LLM terminaram sem timeout. O Ollama informou contexto máximo de 131.072 tokens; os prompts observados consumiram 2.050 tokens.

Caso	Modo	Válidas	Concordância	Aceitação	Mediana da decisão
Grande	Determinístico	3/3	100%	100%	0,000015 s
Grande	Coordinator	3/3	100%	100%	4,985 s
Grande	Sem análises	3/3	100%	100%	5,139 s
Pequeno	Determinístico	3/3	100%	0%	0,000007 s
Pequeno	Coordinator	3/3	100%	0%	3,545 s
Pequeno	Sem análises	3/3	100%	0%	3,560 s

No caso grande, 4 threads obtiveram ganho de 1,673x contra o sequencial e os dois braços LLM reproduziram a admissão determinística. No caso pequeno, os dois braços rejeitaram corretamente em 3/3 e o fallback permaneceu sequencial.

Cada braço LLM processou 12.300 tokens de prompt. As medianas por caso ficaram entre 3,545 s e 5,139 s, contra microssegundos na política. Com três repetições e efeitos de ordem/carregamento, a diferença de latência entre braços não admite interpretação causal.

A campanha original INCOMPLETE foi preservada. Ela revelou um schema que exigia uma propriedade não declarada; após correção e

reforço determinístico, a replicação com os mesmos contextos terminou COMPLETE, com 12/12 respostas LLM válidas. Na amostra, as análises estruturadas não alteraram aceitação ou resultado final. Em 24 cenários controlados, ACCEPT, REJECT e RETUNE acertaram 6/6, enquanto MODIFY foi convertido em REJECT em 6/6, preservando segurança mas não recuperação.

5.5. Campanha exploratória Linux: sete medições por configuração

A campanha principal concluiu 46 configurações, 322 amostras externas e 46 validações de entrada/variante/threads. Foram avaliadas seis combinações de kernel e entrada, com dez curvas de variantes OpenMP. Os baselines sequenciais de Nussinov são compartilhados entre suas três variantes, sem criar novas amostras independentes. A tabela usa sempre quatro threads para tornar o ponto de comparação explícito; a menor mediana pode pertencer a outra configuração.

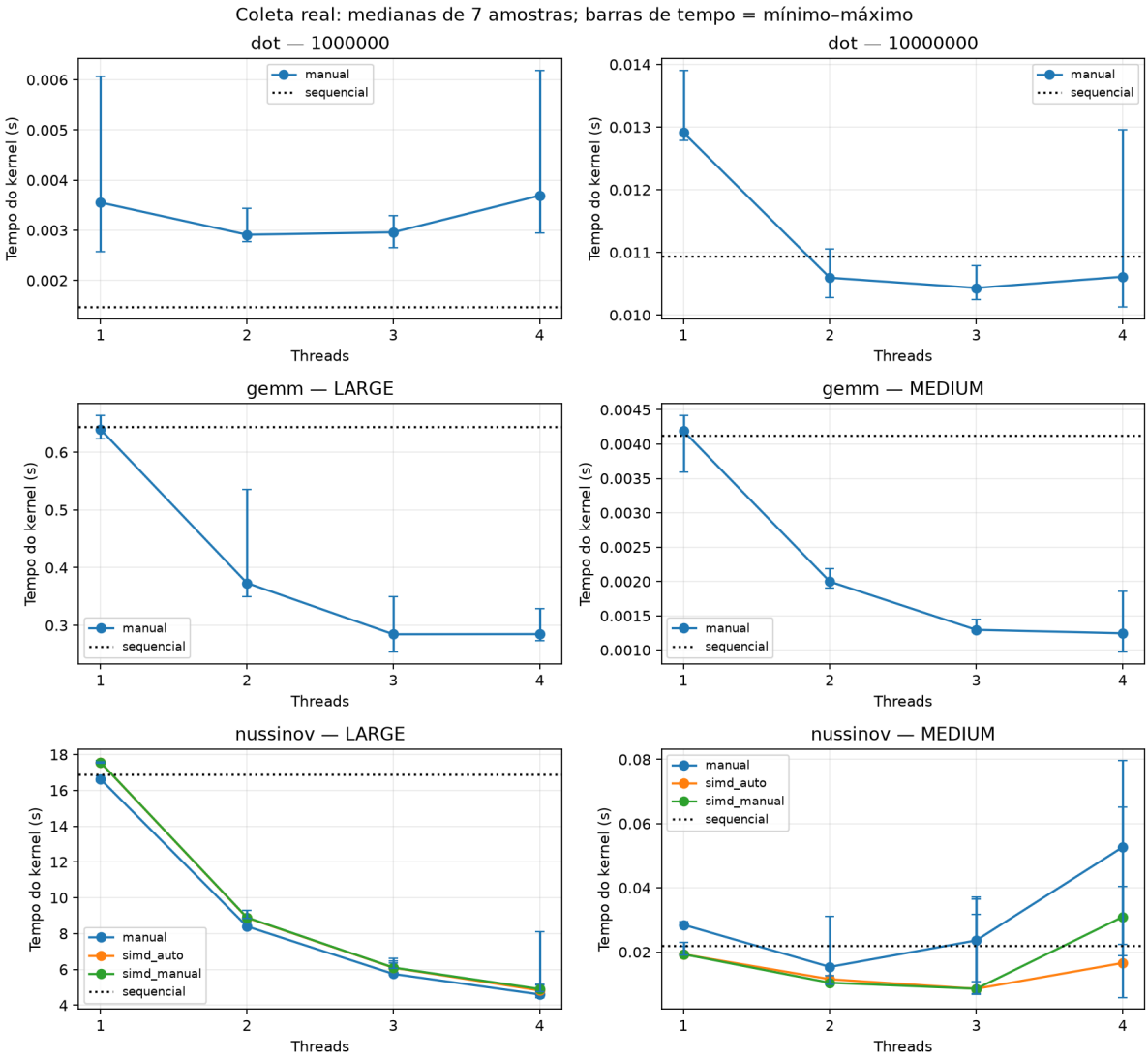
Kernel / entrada	Variante	Tseq (s)	T1 (s)	T4 (s)	S total	S paralelo	E paralela
dot / 1000000	manual	0.001473456	0.003553676	0.003692308	0.399	0.962	0.241
dot / 10000000	manual	0.010942113	0.012910733	0.010611985	1.031	1.217	0.304
gemm / LARGE	manual	0.644089000	0.639590000	0.284841000	2.261	2.245	0.561
gemm / MEDIUM	manual	0.004129000	0.004192000	0.001245000	3.316	3.367	0.842
nussinov / LARGE	manual	16.885718000	16.624249000	4.596830000	3.673	3.616	0.904
nussinov / LARGE	simd_auto	16.885718000	17.560246000	4.820323000	3.503	3.643	0.911
nussinov / LARGE	simd_manual	16.885718000	17.560393000	4.898844000	3.447	3.585	0.896
nussinov / MEDIUM	manual	0.021994000	0.028491000	0.052671000	0.418	0.541	0.135
nussinov / MEDIUM	simd_auto	0.021994000	0.019298000	0.016583000	1.326	1.164	0.291
nussinov / MEDIUM	simd_manual	0.021994000	0.019309000	0.030927000	0.711	0.624	0.156

Dot C++ grande teve menor mediana em 3T (0,010432664 s; ganho total 1,049×). Para N=1 milhão, todas as configurações OpenMP foram mais lentas que o sequencial. GEMM grande apresentou 0,284560 s em 3T e 0,284841 s em 4T: diferença de cerca de 0,10%, compatível com quase empate descritivo, sem prova de significância. Nussinov grande

manual alcançou 3,673× contra o sequencial e 3,616× contra OpenMP 1T, com eficiência de 0,904 em 4T. Em N=500, a versão manual em 4T perdeu para o sequencial; 2T obteve ganho total de 1,432×. Mais threads não foram sistematicamente melhores em entradas médias.

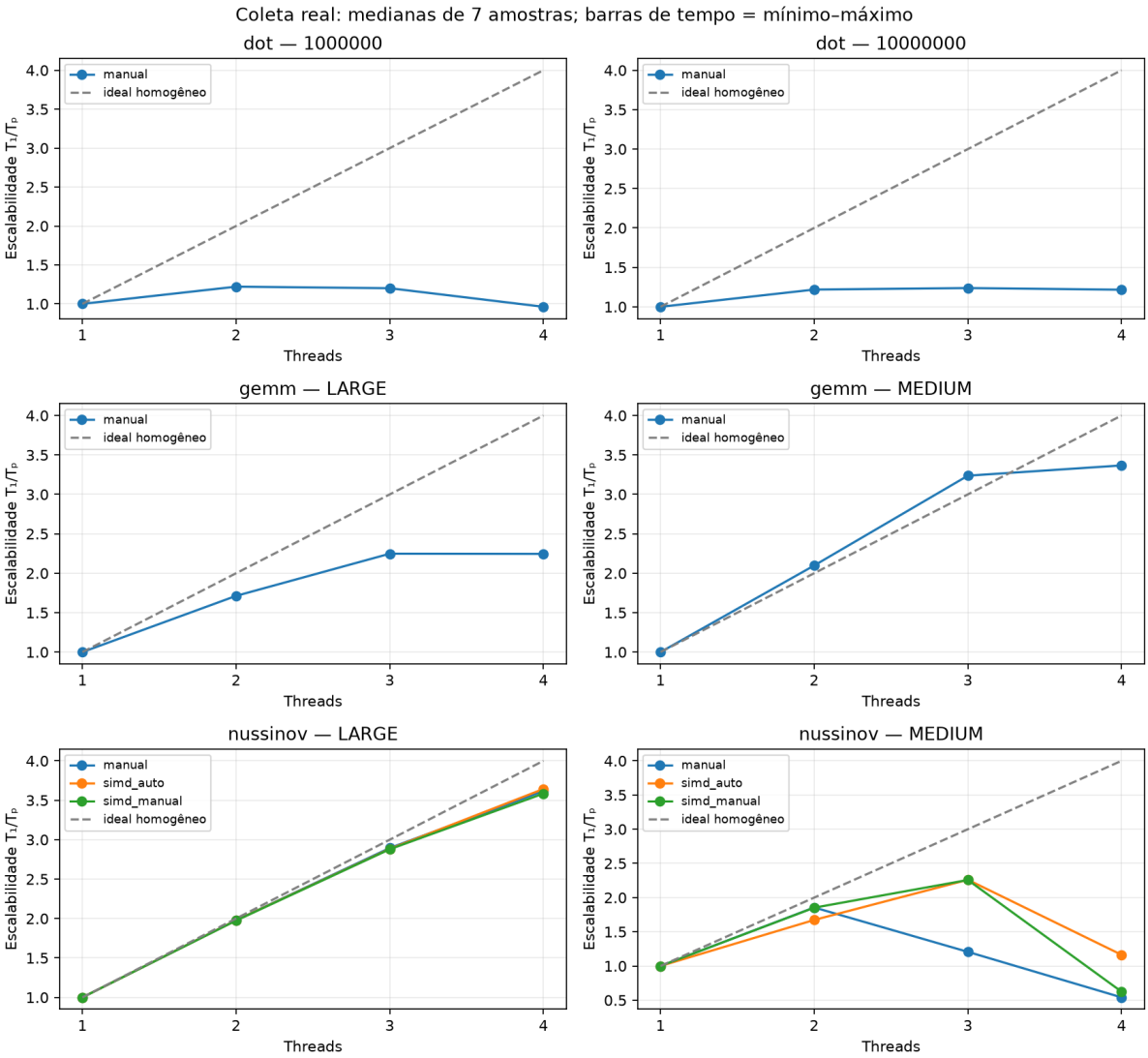
No GEMM médio, T1/Tp excede p em 2T e 3T. Essas razões de medianas não comprovam superlinearidade causal: cache, frequência e variabilidade do baseline podem contribuir. As amostras permanecem disponíveis, sem descartar outliers.

Figura 6 — Tempos reais e amplitude mínimo–máximo da coleta Linux



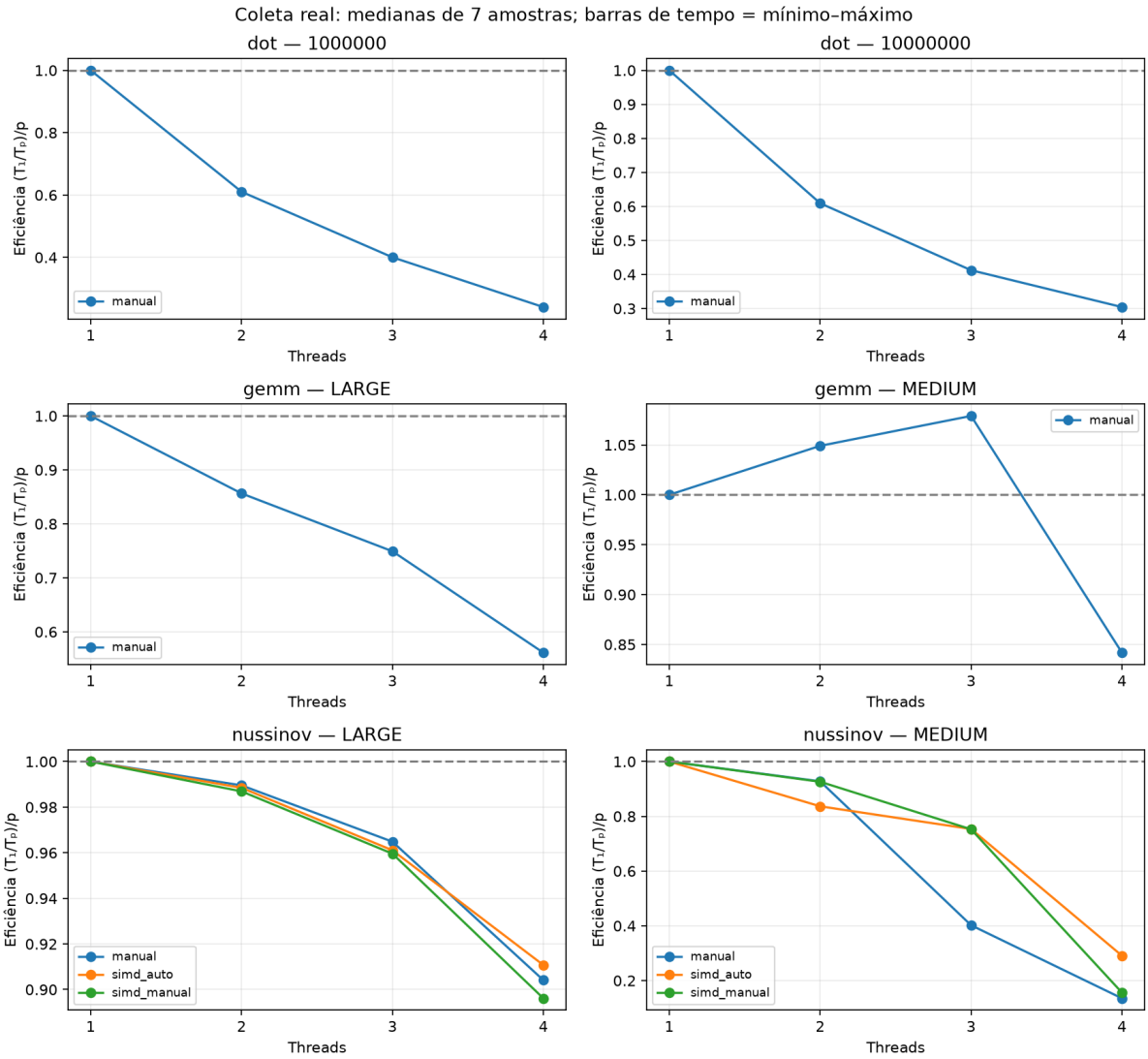
Fonte: elaboração a partir do código e dos resultados do projeto.

Figura 7 — Escalabilidade da mesma variante e entrada no Linux



Fonte: elaboração a partir do código e dos resultados do projeto.

Figura 8 — Eficiência paralela da coleta Linux



Fonte: elaboração a partir do código e dos resultados do projeto.

5.6. Dot C exploratório: cinco medições e efeito do regime de medição

O pipeline foi executado em modo --prepare-only, sem inferência nem promoção, com Clang 18.1.3 e 1T–4T. Os dois casos terminaram PREPARED e passaram pelos gates de corretude. Cada configuração tem cinco medições externas. O caso grande usa 100 chamadas internas de N=10 milhões; o pequeno usa 10.000 chamadas de N=1024. Esses drivers e a distribuição dos valores diferem do Dot C++ manual. A validação do pipeline usa seu driver de N=1 milhão e não constitui um teste de todas as entradas temporizadas.

Caso	Tseq (s)	T1 (s)	T4 (s)	S total 4T	S paralelo 4T	Seleção
large	0.010385878850	0.010864713850	0.008063438350	1.288	1.347	3T; S total 1.264×
small	0.000000889980	0.000001850426	0.000004171944	0.213	0.444	Sem paralelo admissível

No caso grande, 4T teve a menor mediana (0,00806343835 s), mas 3T ficou 1,908% acima dela; a política de quase empate de 2% recomendou 3T, com ganho total de 1,264×, escalabilidade de 1,322× e eficiência de 0,441. O caso pequeno não admitiu paralelo. Essa diferença frente ao Dot C++ mostra que primeira chamada, driver e amortização precisam ser informados. Os resultados não isolam o efeito de um único fator e não são ganhos da LLM.

Os diagnósticos novos do Clang confirmaram o bloqueio por dependência de memória no laço k da versão manual de Nussinov e a vetorização de largura 4 e fator de intercalação 2 nas versões SIMD. Apesar disso, em N=2500, SIMD 1T levou cerca de 17,560 s, contra 16,624 s da versão manual sem a diretiva: a aplicação de SIMD não garante benefício. A seção .text dos executáveis SIMD manual e automático é idêntica nesta compilação LARGE (SHA-256 registrado nos diagnósticos). Diferenças de tempo entre eles não evidenciam uma melhoria do código computacional automático. Os executáveis de diagnóstico são separados daqueles efetivamente medidos.

Fonte: results/scaling/, relatório de verificação, CSVs e diagnósticos de 05/11. As 22 campanhas da matriz anterior permanecem separadas desta coleta.

5.7. Comparação contextual com os registros do M5

A tabela conserva o ponto comum de 4T e usa cada variante com seu próprio baseline OpenMP 1T. Ela compara registros de campanhas distintas, não uma nova campanha pareada entre máquinas. O Dot C possui o mesmo SHA-256 de candidato e driver de benchmark arquivado da promoção M5; GEMM/Nussinov históricos não têm esse vínculo completo. O compilador Linux é Clang 18.1.3, enquanto o acervo M5 registra LLVM Homebrew 23.1.1.

Caso	Linux novo T4 (s)	Linux T1/T4	M5 arquivado T4 (s)	M5 T1/T4	Registro M5
dot automatic_C	0.008063438	1.347	0.001964010	2.465	03/11; gates verificados no Mac
gemm manual	0.284841000	2.245	0.043192000	3.282	setembro; HISTORICAL_UNBOUND
nussinov manual	4.596830000	3.616	0.319542000	2.957	setembro; HISTORICAL_UNBOUND
nussinov simd_manual	4.898844000	3.585	0.209503000	3.015	setembro; HISTORICAL_UNBOUND

Nussinov manual tem maior escalabilidade normalizada no Linux novo, mas menor latência absoluta no registro M5. Logo, escalabilidade relativa e desempenho absoluto respondem a perguntas distintas. O Dot C do estudo pareado M5 de 03/11 obteve 1,673× contra o sequencial, abaixo dos 2,465× da promoção do mesmo dia, reforçando o cuidado com condições e variabilidade de campanha.

No Linux, os contextos originais dos gates Mac contêm caminhos absolutos do outro ambiente, e um contexto em build não foi transferido. A consolidação usa um resumo arquivado explícito, fixado por hash, quando os artefatos locais estão indisponíveis, com rótulo ARCHIVED_SUMMARY_ONLY. Divergências de hash ou contexto inválido continuam sendo erros. Não se reescreveram caminhos nem relatórios históricos para simular uma validação Linux desses gates.

O Nussinov automático histórico continua com hardware desconhecido e não foi incluído como medição M5. As novas curvas M5 nas mesmas entradas, fontes, repetições e política de medição ainda são necessárias para fortalecer esta comparação. Os números acima não isolam ISA, memória ou contribuição da LLM.

5.8. Campanha principal Linux: três aquecimentos e quinze medições

A nova campanha concluiu 60 configurações, 900 amostras externas, 180 aquecimentos e 60 validações aprovadas. As observações pertencem aos cinco algoritmos e seis casos compilados descritos na metodologia. Nenhum tempo anterior foi usado para preencher essa contagem.

Caso	Entrada	Variante	Recomendação	Tp (s)	S total	S paralelo	E paralela
dot-auto	1024	automatic	sequencial	8.78806e-07	1	—	—

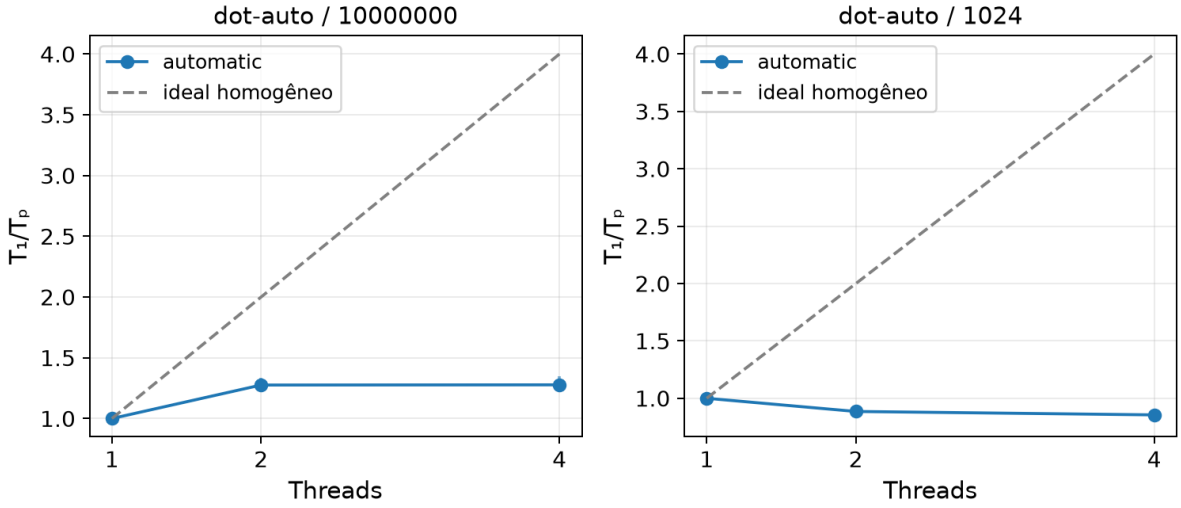
Caso	Entrada	Variante	Recomendação	Tp (s)	S total	S paralelo	E paralela
dot-auto	10000000	automatic	2T	0.00864162	1.25361	1.27581	0.637905
dot	1000000	manual	sequencial	0.0013915	1	—	—
dot	10000000	manual	2T	0.0107116	1.04356	1.23881	0.619404
gemm	LARGE	manual	4T	0.221314	2.84511	2.85401	0.713502
gemm	MEDIUM	manual	4T	0.000967	3.81489	3.97828	0.994571
nussinov	LARGE	manual	4T	5.44933	3.19496	3.20409	0.801022
nussinov	LARGE	simd_auto	4T	5.88327	2.95931	3.09117	0.772792
nussinov	LARGE	simd_manual	4T	5.85374	2.97424	3.1101	0.777524
nussinov	MEDIUM	manual	4T	0.007798	2.70441	3.47794	0.869486
nussinov	MEDIUM	simd_auto	4T	0.005443	3.87452	3.41191	0.852976
nussinov	MEDIUM	simd_manual	4T	0.005448	3.87096	3.41153	0.852882
jacobi-2d	LARGE	manual	2T	2.19022	1.17673	1.18168	0.590838
jacobi-2d	MEDIUM	manual	4T	0.00283	3.60813	3.65265	0.913163
covariance	LARGE	manual	4T	5.60143	2.19285	2.18317	0.545792
covariance	MEDIUM	manual	4T	0.004922	2.17148	2.26249	0.565624

Caso / entrada / variante	Menor mediana OpenMP	Recomendação	Estado
dot-auto / 1024 / automatic	1T; 1.75612e-06 s	sequencial	NO_BENEFICIAL_CONFIGURATION
dot-auto / 10000000 / automatic	4T; 0.00863066 s	2T	SELECTED
dot / 1000000 / manual	2T; 0.00285288 s	sequencial	NO_BENEFICIAL_CONFIGURATION
dot / 10000000 / manual	4T; 0.0105092 s	2T	SELECTED
gemm / LARGE / manual	4T; 0.221314 s	4T	SELECTED
gemm / MEDIUM / manual	4T; 0.000967 s	4T	SELECTED
nussinov / LARGE / manual	4T; 5.44933 s	4T	SELECTED
nussinov / LARGE / simd_auto	4T; 5.88327 s	4T	SELECTED
nussinov / LARGE / simd_manual	4T; 5.85374 s	4T	SELECTED
nussinov / MEDIUM / manual	4T; 0.007798 s	4T	SELECTED
nussinov / MEDIUM / simd_auto	4T; 0.005443 s	4T	SELECTED
nussinov / MEDIUM / simd_manual	4T; 0.005448 s	4T	SELECTED
jacobi-2d / LARGE / manual	2T; 2.19022 s	2T	SELECTED
jacobi-2d / MEDIUM / manual	4T; 0.00283 s	4T	SELECTED

Caso / entrada / variante	Menor mediana OpenMP	Recomendação	Estado
covariance / LARGE / manual	4T; 5.60143 s	4T	SELECTED
covariance / MEDIUM / manual	4T; 0.004922 s	4T	SELECTED

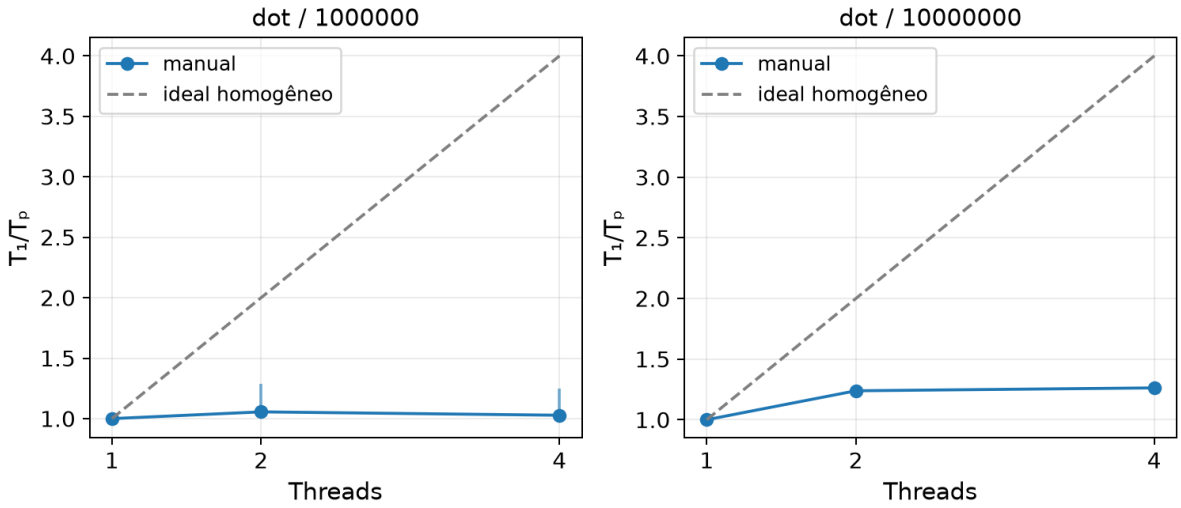
Os valores são arredondados para apresentação; amostras e resumos completos permanecem nos CSVs. A menor mediana OpenMP pode perder para o sequencial e pode diferir da recomendação por quase empate. Os gráficos usam o mesmo baseline 1T da variante e entrada, com ICs bootstrap condicionais.

Figura 9 — Escalabilidade de dot-auto nas duas entradas; barras indicam IC95% bootstrap



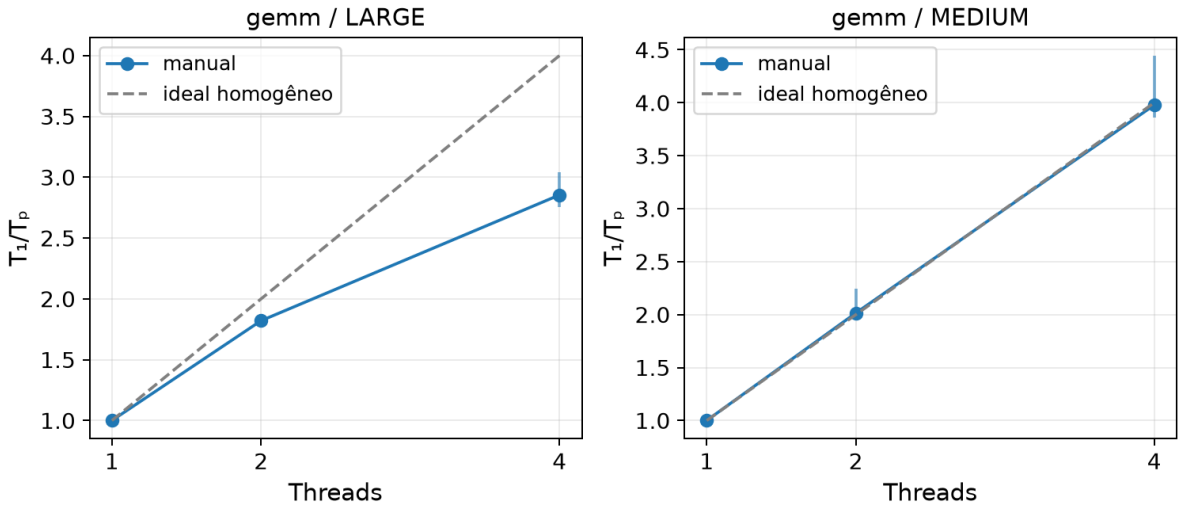
Fonte: elaboração a partir do código e dos resultados do projeto.

Figura 10 — Escalabilidade de dot nas duas entradas; barras indicam IC95% bootstrap



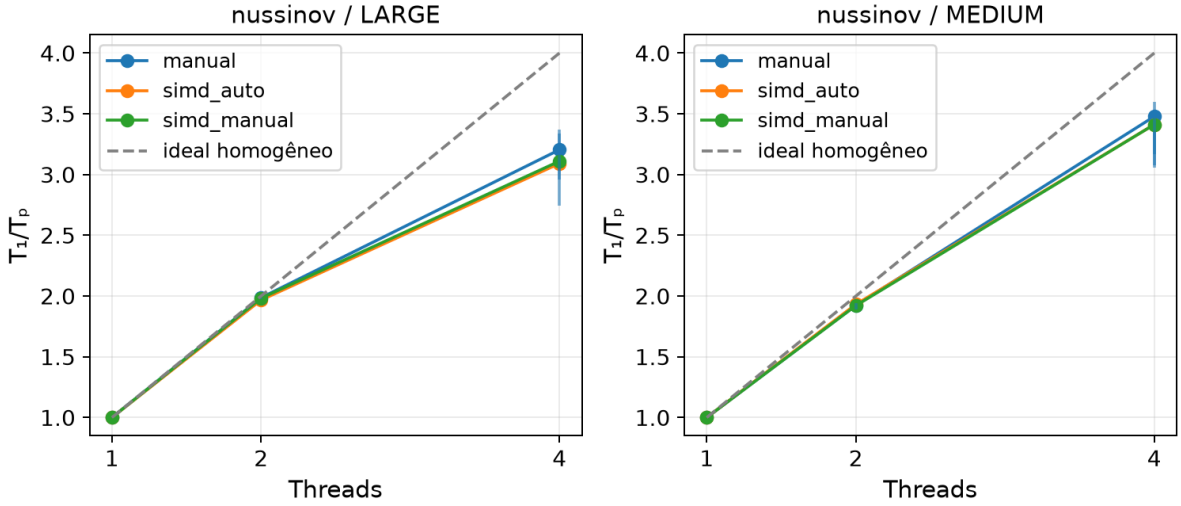
Fonte: elaboração a partir do código e dos resultados do projeto.

Figura 11 — Escalabilidade de gemm nas duas entradas; barras indicam IC95% bootstrap



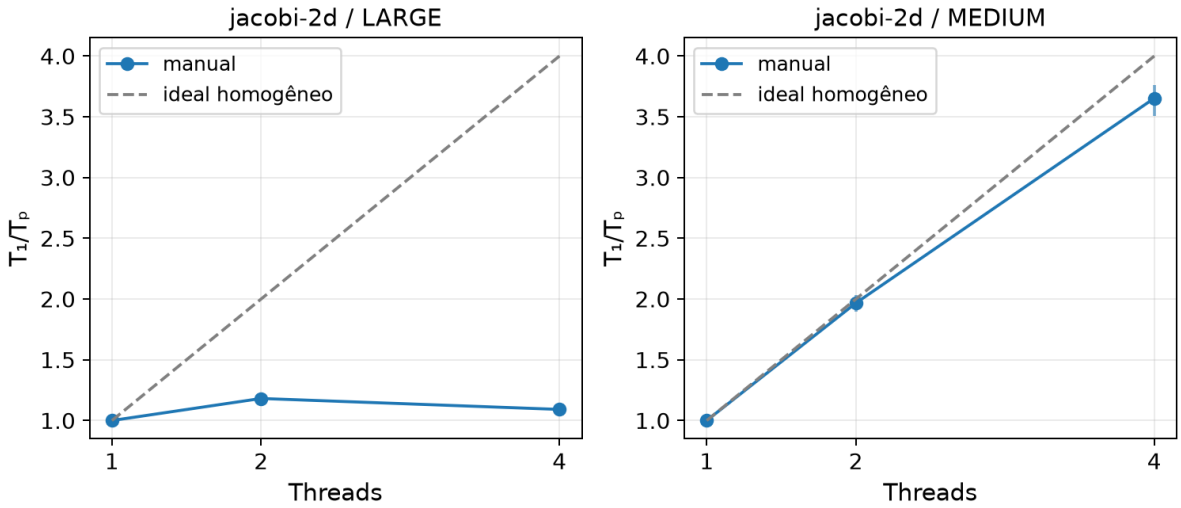
Fonte: elaboração a partir do código e dos resultados do projeto.

Figura 12 — Escalabilidade de nussinov nas duas entradas; barras indicam IC95% bootstrap



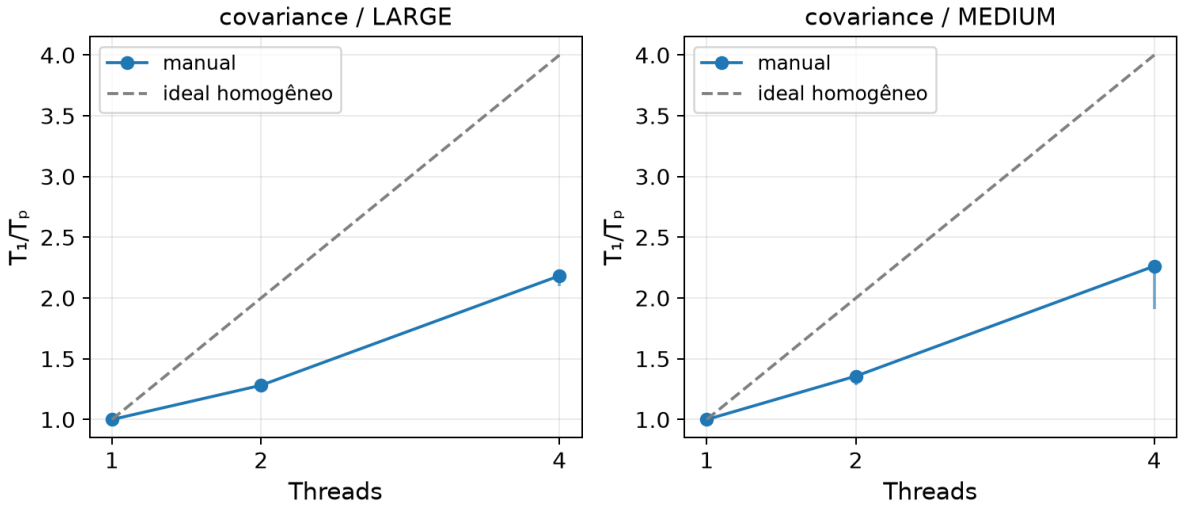
Fonte: elaboração a partir do código e dos resultados do projeto.

Figura 13 — Escalabilidade de jacobi-2d nas duas entradas; barras indicam IC95% bootstrap



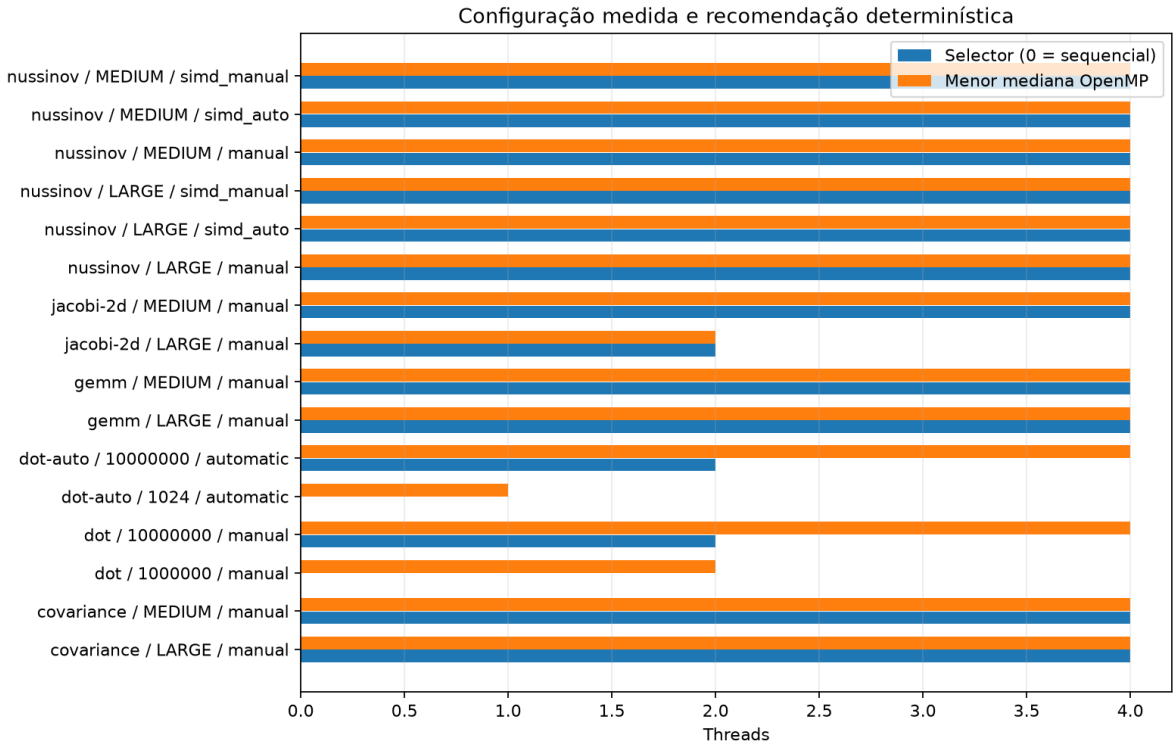
Fonte: elaboração a partir do código e dos resultados do projeto.

Figura 14 — Escalabilidade de covariance nas duas entradas; barras indicam IC95% bootstrap



Fonte: elaboração a partir do código e dos resultados do projeto.

Figura 15 — Configurações mais rápidas e recomendações do Selector



Fonte: elaboração a partir do código e dos resultados do projeto.

A comparação com a nova campanha M5 permanece pendente. As curvas do acervo de setembro e novembro têm protocolos próprios e não completam a nova matriz. Os resultados novos caracterizam códigos e configurações; não uma melhoria decisória da LLM.

6. Discussão e ameaças à validade

Os resultados disponíveis respondem operacionalmente a R1: o protótipo tem mecanismos testados para vincular evidências, bloquear divergências e revalidar o candidato. A resposta é restrita aos contratos e entradas implementados, sem pretensão de uma prova geral de corretude.

Para R2, as campanhas mostram escolhas de configuração distintas e casos em que o paralelismo não supera o sequencial. A contribuição observada pertence à transformação, à implementação da variante e à seleção empírica. O fato de essas etapas produzirem ganho não atribui esse ganho à LLM.

Para R3, o Coordinator reproduziu a política e acrescentou latência sem alterar o speedup, que é igual por construção quando a configuração

aceita é a mesma. A campanha original expôs uma falha estrutural posteriormente corrigida; a replicação final teve respostas válidas. A ablação apresentou os mesmos resultados do contexto completo. Portanto, não foi observado benefício adicional da LLM ou das análises estruturadas nesta amostra. Isso não demonstra inutilidade geral de LLMs para geração de código, tarefa que este experimento não executa.

As principais ameaças à validade são:

- Cobertura funcional: validação numérica para um driver, sem prova formal, detector de races integrado ou cobertura de todas as entradas temporizadas.

- Escopo: estudo do Coordinator em um kernel com dois tamanhos; variantes históricas de outros padrões não ampliam automaticamente essa amostra.

- Medição: ordem fixa das configurações do benchmark, ausência de controle dedicado de frequência/temperatura e possibilidade de ruído do sistema.

- Ablação: remover análises reduz também o tamanho do prompt. Janela de contexto e eventual truncamento devem ser conferidos antes da interpretação.

- Histórico: ausência de vínculos completos entre código, ferramenta, corretude e tempos em 17 campanhas; ambiente atribuído pelo contexto em parte dos registros e amostras ausentes em alguns CSVs.

- Busca não avaliada: um candidato por caso; RETUNE/MODIFY não executam reparo nem permitem medir convergência autônoma.

- Reprodução: caminhos absolutos exigem novas evidências ao mudar de máquina; headers/bibliotecas completos não são capturados pelo contrato atual.

6.1. Interpretação técnica da escalabilidade

Os três kernels exercitam limites diferentes. Dot Product double realiza aproximadamente duas operações aritméticas para cada par de entradas de 16 bytes, ou 0,125 operação por byte do acesso lógico. O modelo Roofline relaciona intensidade operacional, banda de memória e desempenho computacional atingível. Essa conta é uma hipótese explicativa, não uma medição de tráfego DRAM: cache, prefetch e primeira chamada alteram o custo observado. Sem contadores ou um ensaio de banda, uma curva achatada não comprova saturação da memória. Williams, Waterman e Patterson, 2009 (https://escholarship.org/uc/item/5tz795vq).

No GEMM avaliado, a distribuição por linhas evita escrita concorrente no mesmo elemento e conserva o laço interno j contíguo. Existe reuso de B entre linhas, mas o código não incorpora uma biblioteca BLAS otimizada ou uma nova versão blocada. A relação entre conjunto de trabalho, caches e compartilhamento de recursos pode mudar entre as duas entradas. Uma boa razão T1/Tp não demonstra proximidade do pico da máquina ou superioridade frente a BLAS. Os três arrays úteis do GEMM médio ocupam aproximadamente 1,16 MB, enquanto os do grande ocupam 28,96 MB; a tabela Nussinov ocupa 1 MB com N=500 e 25 MB com N=2500. São tamanhos calculados das dimensões e tipos, sem overhead de alocação, e não medições de tráfego. O LLC registrado no Xeon tem 8 MiB, tornando plausível uma mudança de localidade entre entradas; isso exige contadores para confirmação.

Nussinov organiza a computação por d=j−i. A diagonal d possui N−d células; cada célula consulta intervalos menores e executa a redução de máximo no laço k. As diagonais precisam terminar em ordem, mesmo quando as células de uma diagonal são independentes. A região paralela persistente evita recriar workers a cada diagonal, mas conserva N−1 barreiras de final de omp for. Remover a barreira com nowait violaria as dependências. A redução e as barreiras implícitas seguem a especificação OpenMP (https://www.openmp.org/spec-html/5.2/openmp.html).

A variante com acumulador/SIMD separa a redução local da atualização do destino. Ela pode melhorar o código de uma thread, efeito capturado em Tseq/T1, antes do ganho multicore T1/Tp. A presença da diretiva

não comprova que o compilador realizou uma vetorização eficiente. Comparações entre SIMD manual e automaticamente adaptado precisam considerar equivalência do código gerado e dispersão; seus tempos não avaliam uma LLM geradora de código.

No M5, o acervo registra quatro núcleos de desempenho e seis de eficiência. Uma curva com até dez threads não representa dez núcleos homogêneos. Mesmo com OMP_PROC_BIND e OMP_PLACES, o protocolo não comprova quais classes de núcleo executaram cada worker no macOS. A comparação principal deve conservar os pontos comuns de 1T a 4T e analisar 5T a 10T separadamente. Bibliotecas OpenMP, compiladores, cache, memória e energia variam junto com o hardware; os dados não permitem isolar a contribuição individual de cada fator.

6.2. O que a campanha principal acrescenta

No Dot C grande, 4T apresentou a menor mediana, mas 2T ficou apenas cerca de 0,127% acima dela. A política de 2% preferiu duas threads, com ganho total de 1,254× e escalabilidade de 1,276×. No Dot C++ grande, 2T também foi recomendado, com ganho total de apenas 1,044×. O caso C pequeno permaneceu sequencial. Esses resultados mostram que medir quatro threads não implica recomendá-las; também não permitem comparar diretamente a qualidade das transformações C e C++, pois seus drivers e entradas numéricas são distintos.

GEMM grande alcançou escalabilidade de 2,854× e eficiência de 0,714 em 4T, próximas do ganho total de 2,845×. O efeito de transformação em 1T foi pequeno. No conjunto médio, a escalabilidade foi 3,978×. A mudança de conjunto de trabalho entre aproximadamente 1,16 MB e 28,96 MB, frente aos 8 MiB de LLC registrados, fornece uma hipótese de localidade. Não se mediu tráfego ou miss de cache para confirmá-la, nem se comparou esse código simples a uma BLAS otimizada.

Nussinov grande manual atingiu 3,195× contra o sequencial e 3,204× contra OpenMP 1T. As versões SIMD obtiveram ganhos totais menores, de aproximadamente 2,96× e 2,97×. A transformação local não é universalmente benéfica: custo de acesso, redução e código gerado precisam ser medidos na toolchain de cada plataforma. O mesmo princípio vale para as barreiras entre diagonais, cujo custo relativo é

maior quando há pouco trabalho por célula e a largura disponível diminui.

Uma recompilação diagnóstica separada com Clang 18 registrou vetorização do laço de partição das duas variantes SIMD com largura quatro e interleaving dois. A versão manual sem essa redução apresentou diagnóstico de dependência no mesmo laço. Além disso, a extração da seção .text dos executáveis efetivamente medidos mostrou bytes idênticos entre SIMD manual e SIMD adaptado. Logo, a pequena diferença de suas medianas não demonstra vantagem de uma transformação sobre a outra nesta compilação. Os binários diagnósticos não foram medidos nem substituíram os originais; comandos, mensagens e hashes estão em results/multiplatform/diagnostics_linux_xeon_2026-11-05_p15w3/. Vetorização confirmada e ganho multicore são propriedades distintas: no conjunto grande, as duas variantes SIMD ficaram mais lentas que a base manual sem a diretiva.

Jacobi-2D ilustra um comportamento distinto: o conjunto médio foi melhor em 4T, mas o grande teve menor mediana em 2T, com ganho de 1,177×; 4T ficou mais lento. Os dois arrays ocupam aproximadamente 1 MB na entrada média e 27,04 MB na grande. Isso é compatível com uma mudança de pressão sobre memória/cache. A região paralela é persistente, mas há duas barreiras por passo temporal. Sem contadores e uma intervenção de localidade, não se identifica um único gargalo causal.

Covariance grande teve escalabilidade de 1,282× em 2T e 2,183× em 4T. Além da memória, existe uma explicação estrutural verificável: na fase dominante, o laço externo i é dividido estaticamente, mas a quantidade de pares por linha é M−i. Faixas iguais de i recebem volumes diferentes de trabalho. Sob a hipótese de custo uniforme por par, com r=M/p e M divisível por p, a maior faixa realiza rM−r(r−1)/2 pares, e o total é M(M+1)/2. A razão total/maior faixa estima o ganho permitido por essa distribuição, antes de considerar custos adicionais.

Para M=1200, esse modelo de volume fornece aproximadamente 1,334× em 2T e 2,287× em 4T, próximos da ordem de grandeza das razões observadas. No limite contínuo, a expressão é $p^2/(2p-1)$, inferior a p. Esses valores são cálculos do modelo, não medições novas. Eles não

provam que o particionamento explica todo o tempo, pois custos por par, fases menores, cache e frequência também variam. Outra política de schedule não foi medida nesta campanha.

A campanha anterior de sete amostras encontrou perdas no Nussinov médio em 4T; a de quinze amostras encontrou ganho. Os programas não foram otimizados entre as duas coletas. Mudaram aquecimentos, pontos de threads, ordem, sessão e condições não isoladas do sistema. Essa divergência deve ser conservada como limite de repetibilidade, sem juntar amostras ou escolher apenas o resultado mais favorável. Um IC estreito dentro de uma campanha não elimina variabilidade entre campanhas nem torna a frequência/temperatura controlada.

As recomendações distintas do acervo M5 são compatíveis com diferentes recursos e custos, mas os novos pontos M5 ainda não foram coletados. O subconjunto 1T/2T/4T servirá à comparação com códigos e entradas verificados; 6T/8T/10T serão analisados como extensão heterogênea. Comparar os tempos absolutos ou as curvas normalizadas continua sendo comparar plataformas completas, incluindo compiladores e runtimes, sem atribuir diferenças exclusivamente à quantidade de núcleos. A ampliação de algoritmos responde a R2; não amplia a evidência causal de contribuição da LLM em R3.

7. Conclusão compatível com as evidências atuais

Foi implementado um caminho de reestruturação OpenMP para redução escalar que integra análise, geração determinística, validação, medição, seleção e um executor de promoção condicionado a evidências. Sua verificação técnica inclui recusas por inconsistência e revalidação antes da publicação. A organização dos experimentos tornou explícitas as diferenças entre ganho de transformação, escalabilidade e configuração selecionada.

As campanhas locais e históricas sustentam a necessidade de medir configurações e registrar resultados negativos. Não sustentam superioridade de uma arquitetura com vários LLMs sobre geração monolítica. No estudo pareado, o Coordinator não melhorou a decisão admitida: repetiu a política nos dois casos e acrescentou latência. Os gates mantiveram a segurança na campanha original, e o ensaio controlado ainda não demonstrou a decisão MODIFY.

O fechamento restante é consolidar as evidências selecionadas, manter o recorte implementado na redação e finalizar o material de entrega. Ampliação de padrões transformados, recuperação automática de candidatos e suporte a dependências de projetos são extensões, sem promessa de implementação neste TCC.

A coleta complementar de 05/11 mostrou respostas distintas à mesma faixa de threads: Dot com ganho limitado, GEMM grande com quase empate entre 3T e 4T, e Nussinov grande manual com ganho total de 3,673× em 4T. Entradas médias também expuseram perdas e grande dispersão. SIMD foi aplicado pelo compilador, mas não melhorou Nussinov grande nesta toolchain. A comparação contextual com o M5 distingue tempo absoluto e escalabilidade; a nova coleta M5 com protocolo comum continua necessária. Nenhum desses ganhos é atribuído à LLM.

A campanha principal amplia a avaliação para cinco algoritmos, com quinze observações por configuração e critérios explícitos de seleção. Os resultados recomendados do Linux têm ganhos de 1,254× no Dot C, 2,845× em GEMM, 3,195× em Nussinov manual, 1,177× em Jacobi-2D e 2,193× em Covariance nas entradas grandes. A nova coleta M5 e a conferência institucional final permanecem pendentes.

Referências verificadas para esta revisão

Efron, Bradley. Bootstrap Methods: Another Look at the Jackknife. The Annals of Statistics, 7(1), 1–26, 1979. DOI 10.1214/aos/1176344552. Artigo original (https://sites.stat.washington.edu/courses/stat527/s13/readings/ann_stat1979.pdf).

PUCRS. Biblioteca Central Irmão José Otão. Capas padrão e modelo de trabalhos acadêmicos. Consulta em 05/11/2026. Capa pública de graduação (https://biblioteca.pucrs.br/apoio-a-pesquisa/capas-padrao-pucrs/capa-para-tcc/); modelo completo com acesso autenticado (https://biblioteca.pucrs.br/apoio-a-pesquisa/modelos-de-normas-tecnicas-de-documentacao/trabalhos-academicos-abnt/).

Williams, Samuel; Waterman, Andrew; Patterson, David. Roofline: An Insightful Visual Performance Model for Multicore Architectures. Communications of the ACM, 52(4), 65–76, 2009. DOI 10.1145/1498765.1498785. Repositório institucional dos autores

(https://escholarship.org/uc/item/5tz795vq).

OpenMP Architecture Review Board. OpenMP Application Programming Interface, Version 5.2. 2021. Especificação oficial (https://www.openmp.org/spec-html/5.2/openmp.html).

Wu, Qingyun et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. 2023. arXiv:2308.08155v2. Artigo (https://arxiv.org/abs/2308.08155).

Nichols, Daniel; Davis, Joshua H.; Xie, Zhaojun; Rajaram, Arjun; Bhatele, Abhinav. Can Large Language Models Write Parallel Code? 2024. arXiv:2401.12554v3; HPDC '24, DOI 10.1145/3625549.3658689. Artigo (https://arxiv.org/abs/2401.12554).

Pouchet, Louis-Noël. PolyBench/C: the Polyhedral Benchmark suite. Página do projeto. Fonte primária (https://www.cs.colostate.edu/~pouchet/software/polybench/polybench.html).

Apêndice de rastreabilidade e atualização

Fontes internas: catálogo (../experiment_catalog.json), matriz completa (../../results/consolidated/REPORT.md), protocolo dos agentes (../COORDINATOR_EVALUATION.md), execução Linux (../COORDINATOR_EVALUATION_LOCAL.md), campanha Mac (../COORDINATOR_EVALUATION_MAC.md) e reprodução (../EXPERIMENT_REPRODUCTION.md). A conferência dos documentos Word está em LEITURA_DOS_DOCUMENTOS.md (LEITURA_DOS_DOCUMENTOS.md).

A campanha final foi incorporada ao resumo/abstract, §5.4, discussão e conclusão. Não substituir tempos do kernel por latência de inferência. Nome de orientador, template, paginação e exigências formais de entrega não foram inferidos dos arquivos recebidos.

A revisão de 05/11 acrescenta metodologia §4.5, resultados §5.5–§5.7 e discussão técnica §6.1. Fontes adicionais: protocolo multiplataforma (../ESCALABILIDADE_MULTIPLATAFORMA_2026-11-05.md), tabelas, amostras e gráficos novos (../../results/scaling/analysis_linux_xeon_2026-11-05/REPORT.md) e documento de revisão (../revisao_experimentos_2026-11-05/REVISAO.md).

A revisão principal utiliza metodologia de quinze medições (../experiments/MULTIPLATFORM_METHODOLOGY.md), relatório numérico (../experiments/MULTIPLATFORM_REPORT.md) e os arquivos de results/multiplatform/. A fonte original de 32ae80b foi preservada como BASE_MONOGRAFIA_ORIGINAL_32ae80b.md; a base de trabalho anterior também não foi sobrescrita. O gerador scripts/build_multiplatform_material.py produz os novos arquivos Word/PDF e as figuras específicas. O PDF é uma revisão de conteúdo em ReportLab, sem alegação de renderização nativa Office ou aprovação institucional.